RETO 1 · DEL MONOLITO A UN FRONTEND PROFESIONAL

H3 · Frontend estático rediseñado

Flores Nuria · Panel de gestión interna

Frontend independiente en **HTML semántico y CSS** (escrito en Sass) a partir del Design System del Hito 2. Este documento sigue la rúbrica del Hito 3 · RA2 (**CE2A a CE2I**): cada criterio tiene su apartado con evidencias del código, del navegador y de los validadores.

Dato	Valor
Equipo	Equipo 4 · Floristería
Integrantes (GitHub)	dzeledonj-create · dsanchezg-campus · mlizanoj
Ciclo y curso	2.º DAW · Desarrollo de Aplicaciones Web · 2026/2027
Módulo	Diseño de Interfaces Web · RA2 (CE2A–CE2I)
Entrega	H3 · Construid un frontend independiente · versión revisada 20/10/2026
Repositorio	github.com/dzeledonj-create/flores-nuria_main

Índice

1. Introducción 3
1.1 Cómo ejecutar el proyecto y dónde está cada evidencia 3
2. Punto de partida: el frontend antes del H3 5
3. CE2A · Etiquetas HTML y su presentación con CSS 6
3.1 HTML estructurado y semántico 6
3.2 Presentación de elementos HTML con CSS 6
3.3 Separación entre estructura y presentación 7
3.4 Coherencia entre vistas 7
4. CE2B · Estilos directos: práctica inline frente a CSS externo 9
4.1 Cascada y prioridad del estilo inline 10
4.2 Versión con CSS externo y comparación 11
4.3 Conclusión técnica 11
5. CE2C · Hojas externas y estilos globales 12
5.1 Organización de los archivos 12
5.2 Estilos globales alineados con el Design System 13
5.3 Vinculación externa en todas las vistas 14
5.4 Compilación reproducible 14
5.5 Mantenimiento: un cambio global sin tocar las vistas 14
6. CE2D · Hoja de estilos alternativa: tema oscuro 16
6.1 Cómo activar cada versión 16
6.2 Comprobación de las dos versiones 17
7. CE2E · Cascada, herencia y redefinición 19
7.1 Depuración con las herramientas del navegador 19
7.2 Otros casos de cascada y herencia 21
7.3 Redefinición controlada de variantes y estados 21
7.4 Selectores y especificidad sin !important 22
8. CE2F · Propiedades CSS 23
8.1 Distribución con Grid y Flexbox 23
8.2 Combinaciones de propiedades explicadas 23
8.3 Aplicación del diseño del Hito 2 24
9. CE2G · Clases CSS 25
9.1 Convención de nombres 25
9.2 Reutilización en las vistas 25
9.3 Variantes y estados 25
10. CE2H · Validación CSS 26
10.1 Resultados antes y después 26
10.2 Errores detectados y corregidos 27
10.3 Revalidación y advertencias que se mantienen 28
11. CE2I · Guía de estilo actualizada 30
11.1 Cambios respecto al Hito 2 32
11.2 Cómo usar la guía para ampliar la interfaz 33
12. Diseño responsive 34
13. Validación HTML y accesibilidad 35
14. Ecodiseño: mediciones antes y después 36
15. Git y documentación 37
16. Relación con la rúbrica 38
17. Conclusión 39

1. Introducción

En el Hito 2 diseñamos el **Design System** de Flores Nuria. En este Hito 3 lo hemos convertido en un **frontend independiente**: 19 vistas en HTML semántico que solo usan hojas de estilo externas generadas con Sass. No hay JavaScript: menús, pop-ups, paneles y errores de formulario se resuelven con HTML y CSS.

Criterio	Qué demostramos	Apartado
CE2A · Etiquetas HTML	HTML semántico en las 19 vistas, presentación de elementos nativos con CSS, separación total y coherencia entre vistas	3
CE2B · Estilos directos	Práctica con un componente inline y su versión con clases, pruebas de prioridad y conclusión	4
CE2C · Hojas externas	Estilos globales, enlaces en todas las vistas, organización en parciales y mantenimiento con tokens	5
CE2D · Hojas alternativas	Tema oscuro como hoja alternativa (theme-dark.css) sobre el mismo HTML, cómo activarla y comprobarla	6
CE2E · Redefinición	Casos reales de cascada, herencia, especificidad y depuración con DevTools	7
CE2F · Propiedades CSS	Propiedades por categoría, Flexbox/Grid y aplicación del diseño del Hito 2	8
CE2G · Clases CSS	Sistema BEM, reutilización, variantes y estados	9
CE2H · Validación CSS	W3C CSS Validator y Stylelint antes y después de corregir	10
CE2I · Guía de estilo	Guía de estilo v1.1 actualizada, en HTML con el CSS real	11

> **Cómo se han obtenido las evidencias.** El estado ANTES es el commit `2c6fe9b`; el estado DESPUÉS es el código actual del repositorio. Las capturas de pantallas son del navegador (Chromium) a 1440, 768 y 390 px. Las vistas del editor y del panel de DevTools están generadas a partir de los archivos reales del repositorio y de los datos reales del inspector de Chrome (protocolo DevTools). La validación CSS usa el **W3C CSS Validator** oficial y **Stylelint**; la HTML, el **Nu Html Checker** del W3C.

1.1. Cómo ejecutar el proyecto y dónde está cada evidencia

1. Clonar el repositorio y abrir la carpeta en Visual Studio Code.
2. `npm install` (solo la primera vez: instala Sass y Stylelint).
3. `npm run build:css` compila las cuatro hojas; `npm run lint:css` pasa el linter.
4. Abrir `index.html` con Live Server o con XAMPP (`http://localhost/flores-nuria_main/`): redirige a `resources/views/login.html`.

Evidencia	Archivo en el repositorio
Vistas de la aplicación	resources/views/ (vistas de una pantalla) y subcarpetas productos/, pedidos/, pagos/ y proveedores/ (listado + alta); plantillas en resources/views/parciales/
Código fuente de los estilos	resources/scss/ (abstracts, base, themes, layout, components, docs)
Hoja principal y alternativa	public/css/main.css · public/css/theme-dark.css
Práctica CE2B	docs/practicas/inline-vs-externo.html
Guía de estilo v1.1 (CE2I)	docs/guia-estilo.html
Configuración del linter (CE2H)	.stylelintrc.json · package.json (lint:css)

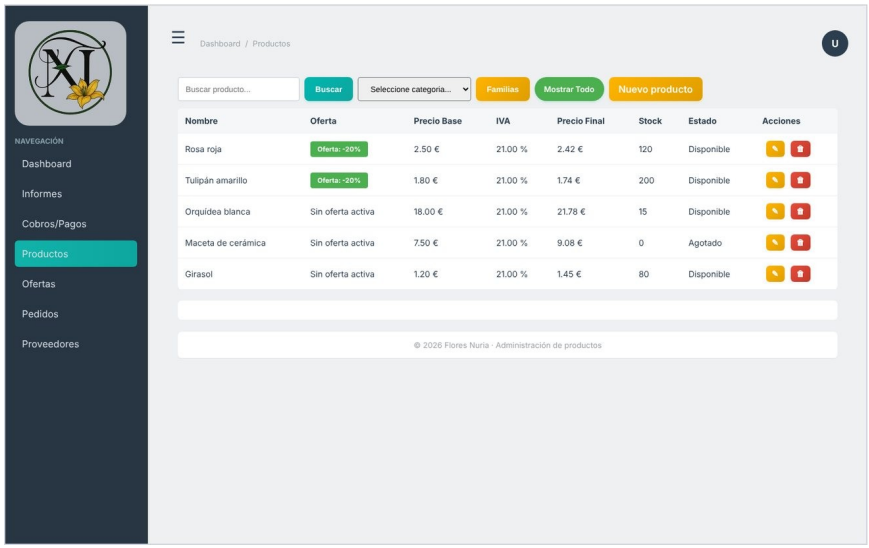
Evidencia	Archivo en el repositorio
Documentación	README.md · CHANGELOG.md · docs/

2. Punto de partida: el frontend antes del H3

Analizamos el frontend del repositorio antes de reconstruirlo (19 vistas y un único `styles.css` de 1.143 líneas):

Problema medido	Dato
Errores de HTML en el Nu Html Checker (W3C)	26 errores y 219 avisos
`<main>` dentro de `<section>` · páginas sin `<h1>`	18 · 18 de 19
`<section>` usada como contenedor genérico	204 secciones (190 avisos)
Campos con `<label>` sin `for`	41 de 44
Colores escritos a mano en la hoja de estilos	145 valores · 72 colores distintos
Clases de utilidad en el HTML (`d-flex`, `mt-5`, `px-3-py-1`…)	44 clases distintas
Estilos en línea y colores en atributos SVG	3 `style=""` · 25 `fill="#…"`
Problemas de accesibilidad (axe) en 8 pantallas	79 elementos (7 críticos)
Móvil: la navegación desaparece y la página desborda	Productos a 675 px en un móvil de 390 px

ANTES · escritorio



ANTES · móvil (desborda a 675 px)

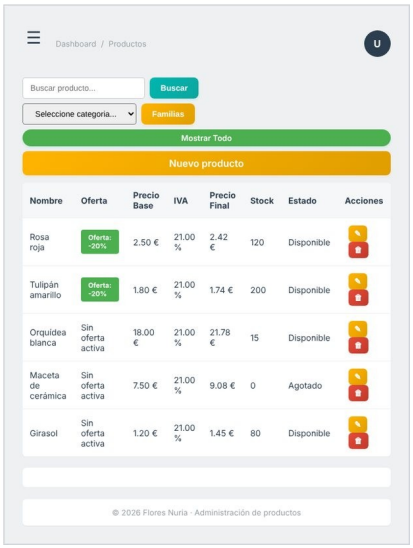


Figura 1. Productos antes del H3: botones de colores distintos, estados solo en texto e iconos emoji; en móvil la tabla desborda.

3. CE2A · Etiquetas HTML y su presentación con CSS

3.1. HTML estructurado y semántico

Todas las vistas comparten el mismo esqueleto. Cada zona usa el elemento que le corresponde:

Esqueleto común (resources/views/productos/products.html, simplificado)

```
<a class="skip-link" href="#main">Saltar al contenido</a>
<div class="app">
  <nav class="sidebar" id="sidebar" aria-label="Principal">…</nav>
  <div class="app__main">
    <header class="topbar">
      <a class="topbar__menu" href="#sidebar" aria-label="Abrir menú de navegación">☰</a>
      <nav aria-label="Migas de pan"><ol class="breadcrumbs">…</ol></nav>
      <details class="user-menu"><summary>…</summary>…</details>
    </header>
    <main class="app__content" id="main" tabindex="-1">
      <header class="page-header"><h1 class="page-header__title">Productos</h1>…</header>
      <form class="toolbar__form" role="search">…</form>
      <table class="table"><caption>…</caption><th scope="col">…</th>…</table>
      <nav class="pagination" aria-label="Paginación de productos">…</nav>
    </main>
    <footer class="app__footer">© 2026 Flores Nuria</footer>
  </div>
</div>
```

Elemento	Antes	Después	Uso
`<nav>` con aria-label	36	41	Menú principal, migas y paginación
`<header>` / `<main>` / `<footer>`	18 / 18 / 18	44 / 19 / 19	Barra superior, cabeceras, contenido principal y pie
`<h1>`	1	19	Un título principal por vista
`<section>`	204	27	Solo bloques con título propio
`<label for>`	3	51	Todos los campos con etiqueta visible
`<table>` + `<caption>` + `th scope`	0 / 0	10 / 57	Tablas de datos accesibles
`<time datetime>`	0	11	Fechas legibles y con valor de máquina
`<details>` / `<summary>`	0	23	Menú de usuario, detalle de venta, datos de gráficos
`<fieldset>` + `<legend>`	0	3	Grupos de productos en pedidos y ofertas

3.2. Presentación de elementos HTML con CSS

No nos limitamos a dar color: cambiamos con criterio cómo se ven y se comportan elementos nativos, sin cambiar su semántica.

Elemento nativo	Qué modificamos	Dónde
`<table>`	Filas alternas, hover, columnas numéricas a la derecha y, en móvil, cada fila pasa a ser una tarjeta (`display: block` + `content: attr(data-label)`)	components/_table.scss

Elemento nativo	Qué modificamos	Dónde
`<select>`	`appearance: none` y flecha dibujada con dos `linear-gradient` que usan tokens (cambia con el tema)	components/_field.scss
`<input type="checkbox">`	Casilla con `accent-color` de marca; con `role="switch"` se convierte en interruptor con `appearance: none` y `::after`	components/_field.scss
`<details>` / `<summary>`	Se quita el triángulo (`list-style: none`) y el summary se convierte en el avatar del menú o en el enlace «Ver»	components/_dropdown.scss, _modal.scss
`<fieldset>` / `<legend>`	Se anulan borde y relleno del navegador; la legend pasa a ser un título de sección	components/_field.scss
`<a>`	Subrayado con `text-underline-offset`, grosor en hover; como `.btn` se ve como un botón	components/_link.scss, _button.scss
`<svg>` (iconos y gráficos)	Color por herencia (`currentColor`) y barras con `fill: var(--color-chart-1)`	components/_chart.scss

3.3. Separación entre estructura y presentación

Comprobado en las 19 vistas: **0 atributos `style`, 0 bloques `<style>` y 0 `<script>`**. Las 19 enlazan `public/css/main.css` y la hoja alternativa `theme-dark.css`. El único archivo del proyecto con estilos inline es la práctica del CE2B (apartado 4), que está fuera de la aplicación precisamente para comparar.

3.4. Coherencia entre vistas

Los elementos equivalentes tienen el mismo aspecto en todas las pantallas porque son las mismas clases: barra lateral, barra superior, cabecera de página, barra de acciones, tabla, paginación y formularios. Las plantillas de referencia están en `resources/views/parciales/`.

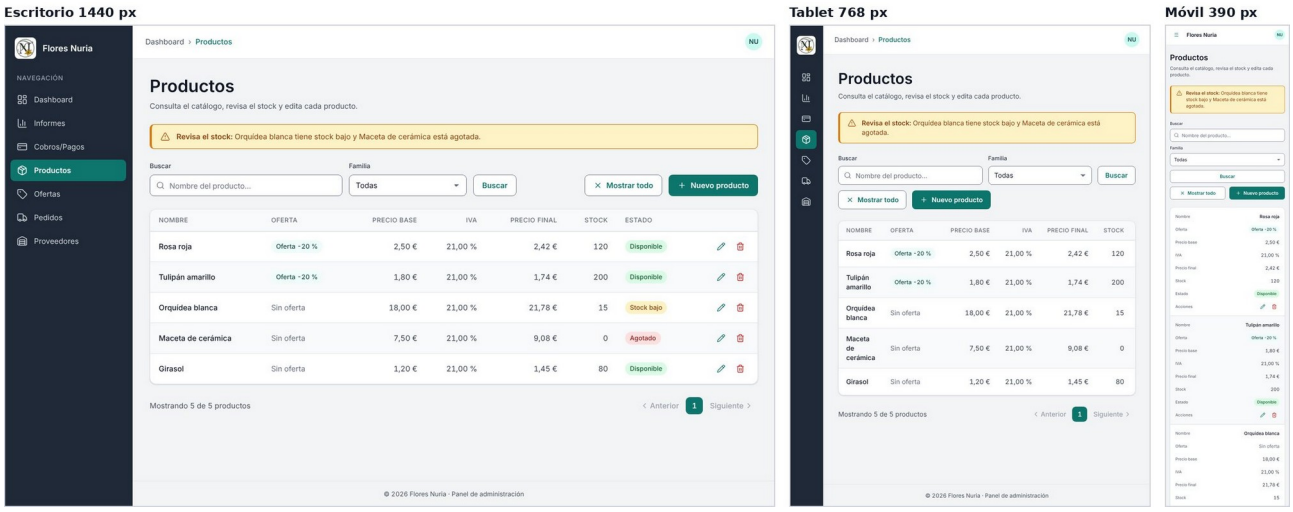


Figura 2. Plantilla de listado en Productos: misma estructura en escritorio, tablet y móvil.

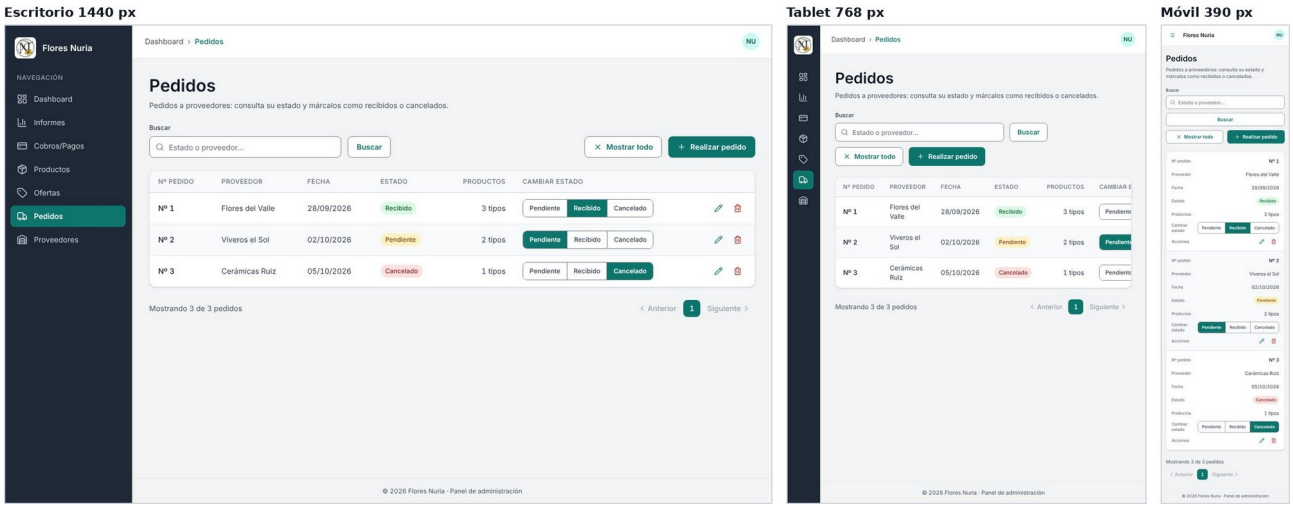


Figura 3. La misma plantilla en Pedidos: cabecera, barra de acciones, tabla y paginación idénticas.

4. CE2B · Estilos directos: práctica inline frente a CSS externo

Para comparar, hemos rehecho la tarjeta KPI «Ingresos» del Dashboard de dos formas en una página de pruebas, `docs/practicas/inline-vs-externo.html`: con **26 declaraciones inline en 4 atributos style** y con **4 clases del Design System**.

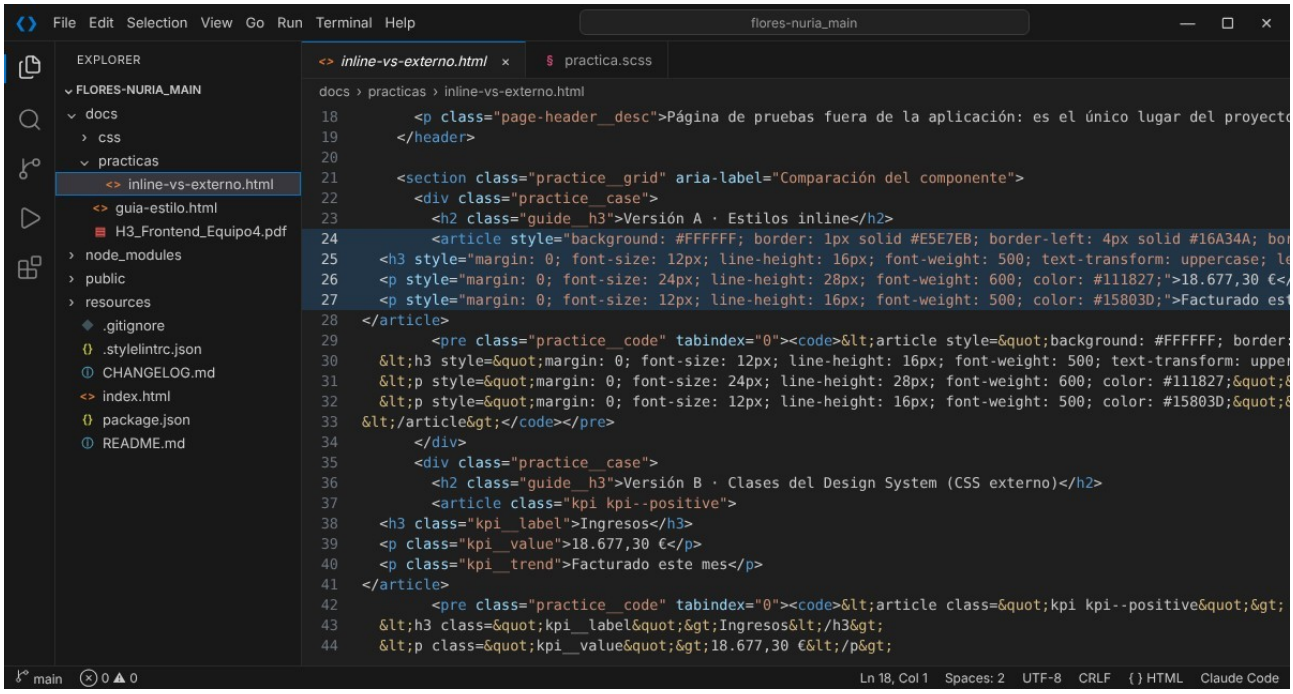


Figura 4. Versión A: el componente con atributos style (líneas 24–27).

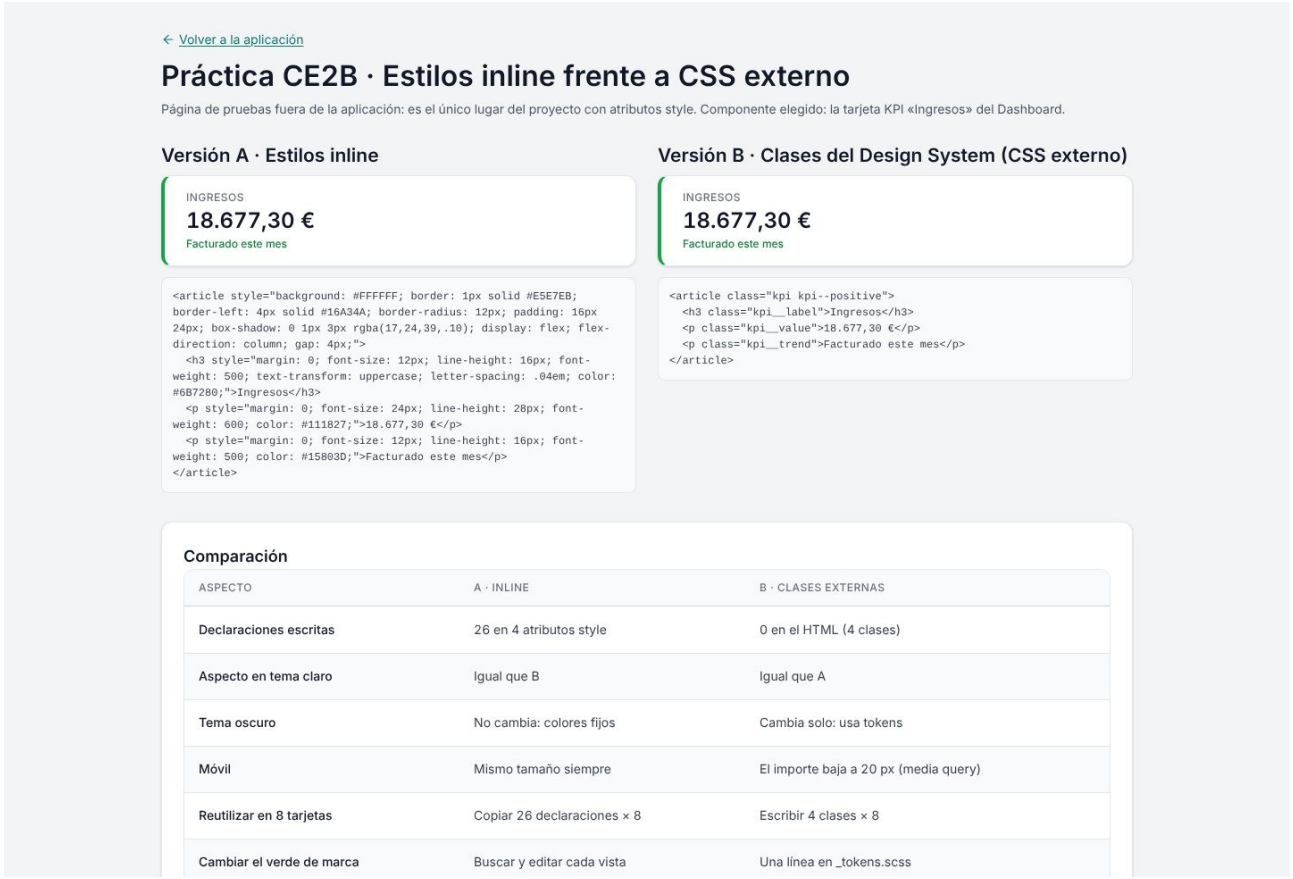


Figura 5. Resultado en el navegador: la versión inline (A) y la versión con clases (B) son idénticas en tema claro.

4.1. Cascada y prioridad del estilo inline

Probamos qué gana en cada caso. El navegador decide en este orden: **1) importancia** (`!important`), **2) origen** (el atributo style va por encima de las hojas), **3) especificidad** del selector y **4) orden** de aparición.

Caso	Código	Gana	Por qué
1 · Inline frente a clase	`class="badge badge--success" style="background:#FDE68A"`	Inline (amarillo)	El atributo style supera a cualquier selector de la hoja, aunque `.badge--success` sea más específica que `.badge`.
2 · `!important` en la hoja	`.prueba-importante{background:#DBEAFE !important}` + style normal	Hoja (azul)	La importancia se compara antes que el origen: una declaración importante de la hoja supera a una inline normal.
3 · Inline `!important`	style con `!important` + la misma clase	Inline (rojo)	Si las dos son importantes, vuelve a mandar el atributo style. Es casi imposible de sobrescribir.
4 · Estados	`<a style="…">` frente a `<a class="link">`	Solo la clase reacciona	En un atributo style no se pueden escribir `:hover`, `:focus-visible`, media queries ni `::before`.

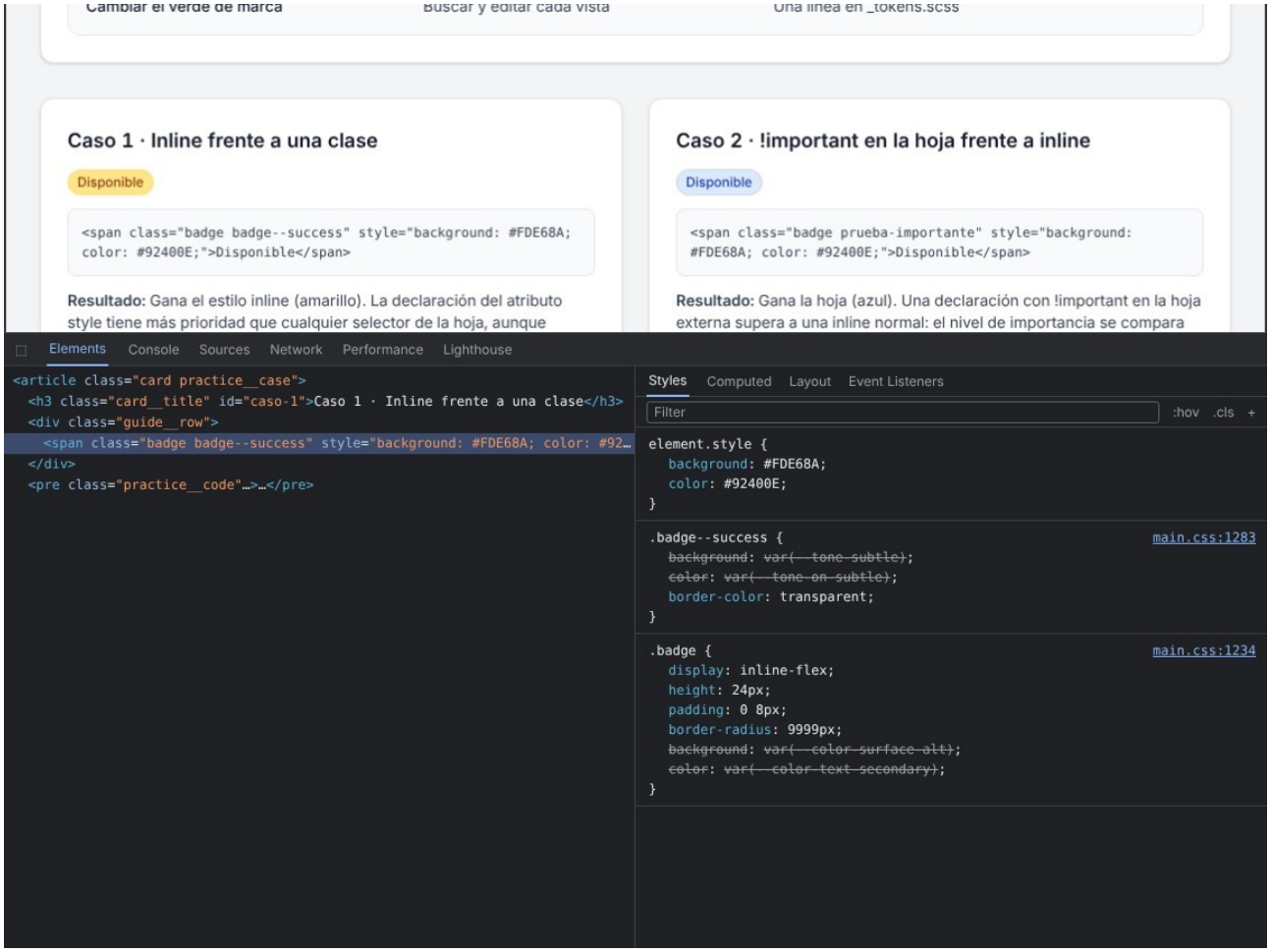


Figura 6. Caso 1 en el panel Estilos de DevTools: element.style está arriba y tacha background y color de .badge--success y .badge.

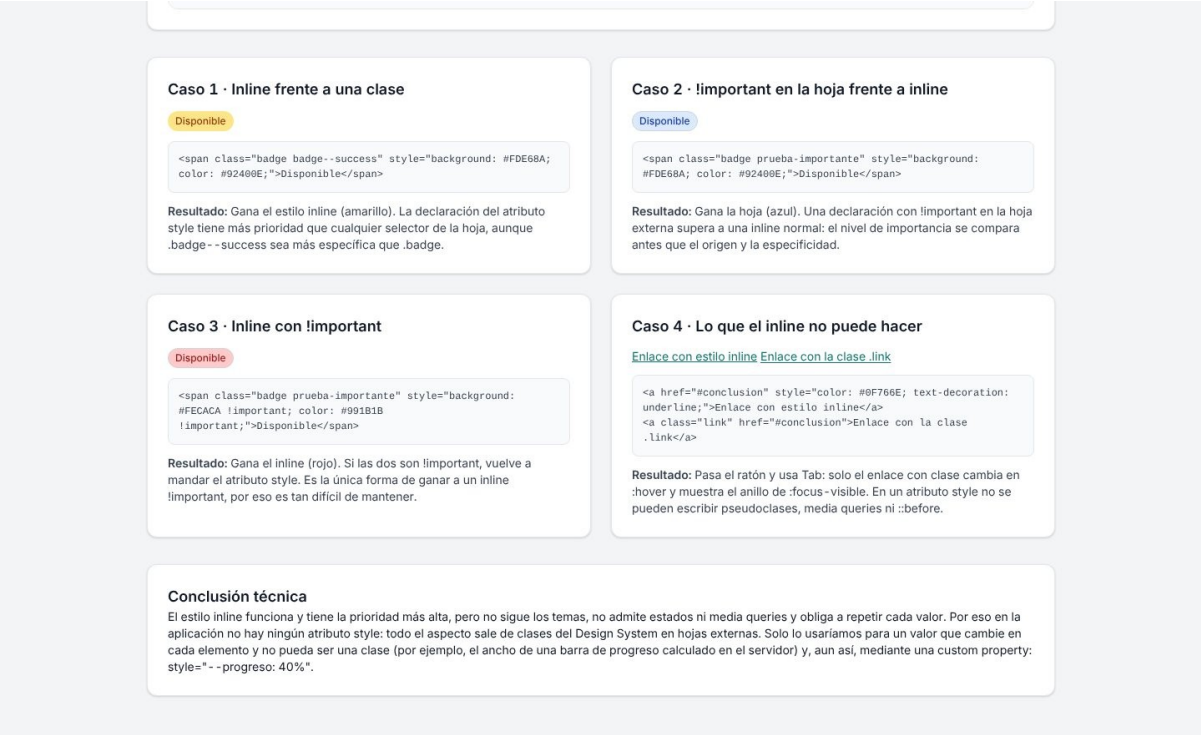


Figura 7. Los cuatro casos en el navegador, con el código y el resultado de cada uno.

4.2. Versión con CSS externo y comparación

Aspecto	A · Inline	B · Clases externas
Declaraciones en el HTML	26 en 4 atributos	0 (4 clases)
Tema oscuro	No cambia: colores fijos	Cambia solo: usa tokens
Móvil	Mismo tamaño siempre	El importe baja a 20 px (media query)
Reutilizar en las 8 tarjetas KPI	Copiar 26 declaraciones × 8	Escribir 4 clases × 8
Cambiar el verde de marca	Editar cada vista	Una línea en _tokens.scss

Tema claro



Tema oscuro: la versión inline no cambia



Figura 8. Con el tema oscuro del sistema, la tarjeta inline sigue blanca (izquierda de cada captura) y la de clases se adapta.

4.3. Conclusión técnica

> El estilo inline tiene la prioridad más alta, pero **no sigue los temas, no admite estados ni media queries y obliga a repetir cada valor**. Por eso la aplicación no tiene ningún atributo style. Solo lo usaríamos para un dato que cambie en cada elemento y no pueda ser una clase (por ejemplo, el ancho de una barra de progreso que venga del servidor), y aun así pasándolo como custom property: `style="--progreso: 40%"`.

5. CE2C · Hojas externas y estilos globales

5.1. Organización de los archivos

Seguimos la organización de Laravel: **`public/` contiene solo lo que carga el navegador** (CSS compilado, fuentes, imágenes y datos) y **`resources/` el código fuente** (el SCSS y las vistas). El SCSS de `resources/scss/` se compila a `public/css/` (la aplicación) y a `docs/css/` (guía y práctica).

Estructura del repositorio

```
flores-nuria_main/
├── index.html          → redirige a resources/views/login.html
├── public/             → lo que carga el navegador
│   ├── css/            → main.css (principal) y theme-dark.css (alternativa)
│   ├── fonts/  img/  data/
├── resources/
│   ├── scss/           → código fuente de los estilos
│   └── views/          → vistas: raíz + productos/ pedidos/ pagos/ proveedores/ +
parciales/
├── docs/               → guía de estilo, práctica CE2B y esta memoria
└── package.json  .stylelintrc.json  README.md  CHANGELOG.md
```

Así se veía el proyecto en nuestro Visual Studio Code durante el H3, antes de pasar el SCSS y las vistas a `resources/`:

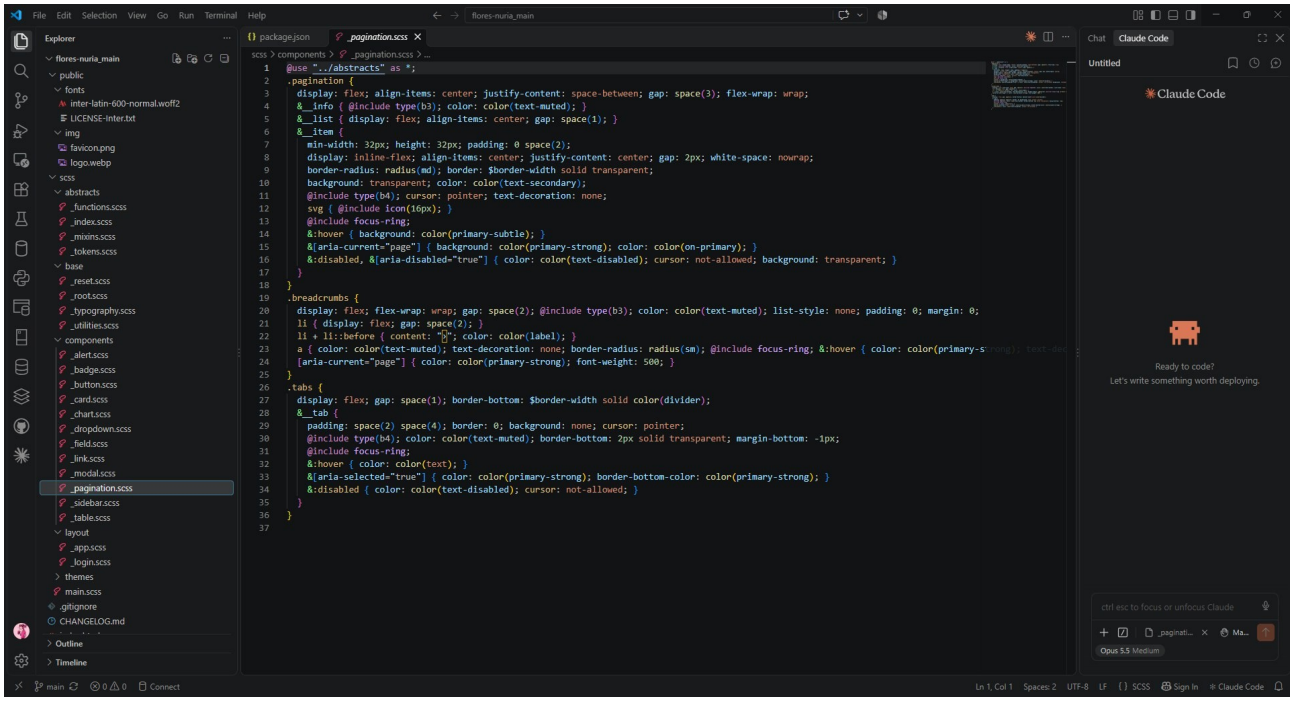


Figura 9. Nuestro VS Code durante el H3: public/ (fonts, img) y el SCSS separado en abstracts, base, components, layout y themes, con un parcial por componente.

Carpeta	Responsabilidad	Genera CSS
resources/scss/abstracts	Tokens, funciones y mixins	No
resources/scss/base	Custom properties, @font-face, reset, tipografía y utilidades mínimas	Sí
resources/scss/themes	_dark (dentro de main.css) y alt-dark.scss (hoja alternativa)	Sí
resources/scss/layout	Esqueleto de la aplicación y login	Sí

Carpeta	Responsabilidad	Genera CSS
resources/scss/components	12 parciales: un archivo por componente	Sí
resources/scss/docs	Estilos solo de la guía y de la práctica	Sí, en docs/css/

Las vistas también están organizadas: las que tienen listado y formulario de alta van juntas en su carpeta (`productos/`, `pedidos/`, `pagos/`, `proveedores/`), las de una sola pantalla en `resources/views/` y las plantillas en `resources/views/parciales/`. Enlazan los recursos con rutas relativas (`../../public/…` o `../../../public/…`), así funcionan con Live Server y con XAMPP. Al pasar a Laravel serán plantillas Blade que enlazarán con `asset('css/main.css')`, porque allí `public/` es la raíz web.

5.2. Estilos globales alineados con el Design System

Todo valor visual sale de `_tokens.scss`: 55 colores en cada tema, escala tipográfica, espaciados, radios, sombras y breakpoints. `_root.scss` los publica como custom properties y `_reset.scss` fija las reglas globales del body (fuente, color, fondo, cifras tabulares).

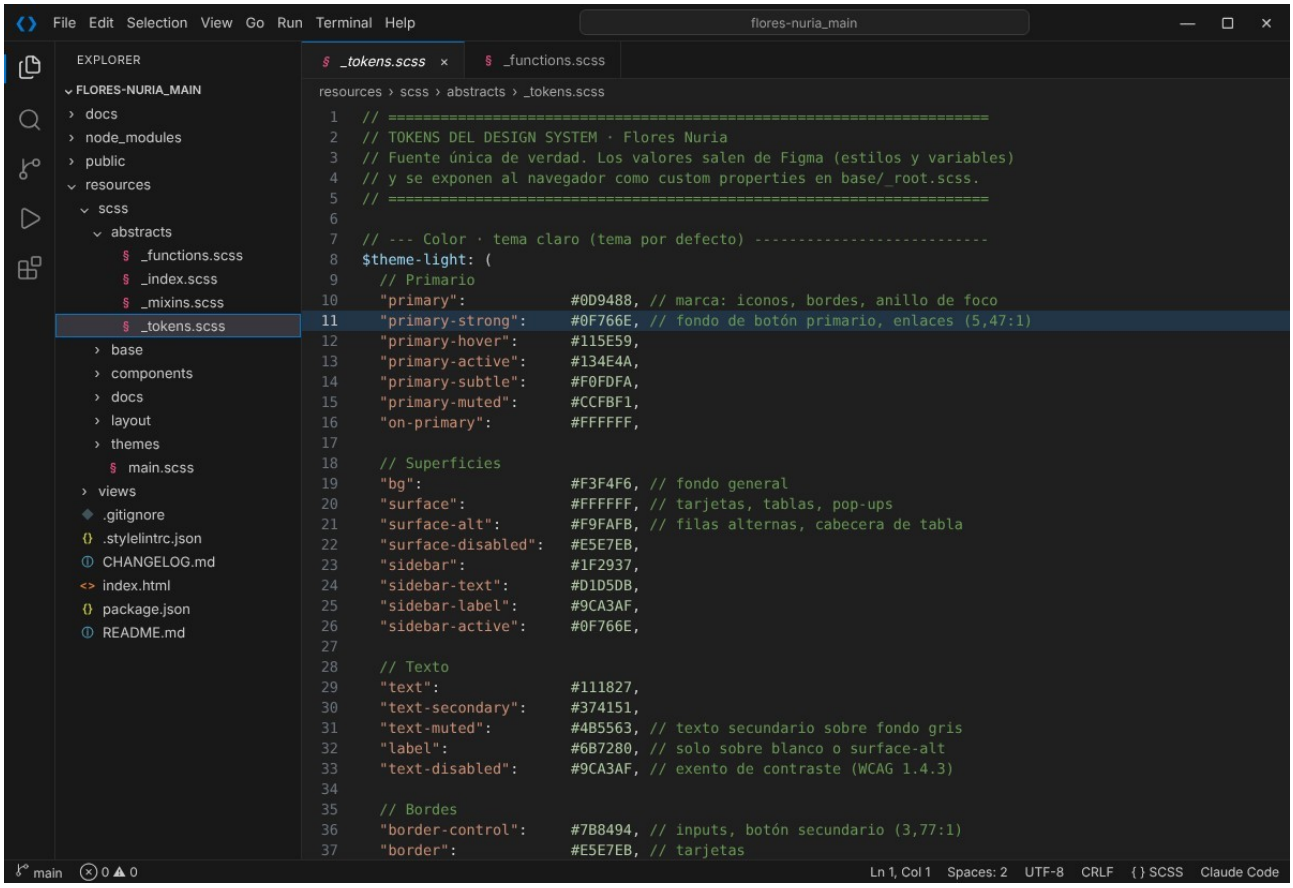


Figura 10. Los tokens del tema claro: primary-strong (línea 11) es el verde de los botones, enlaces y opción activa.

5.3. Vinculación externa en todas las vistas

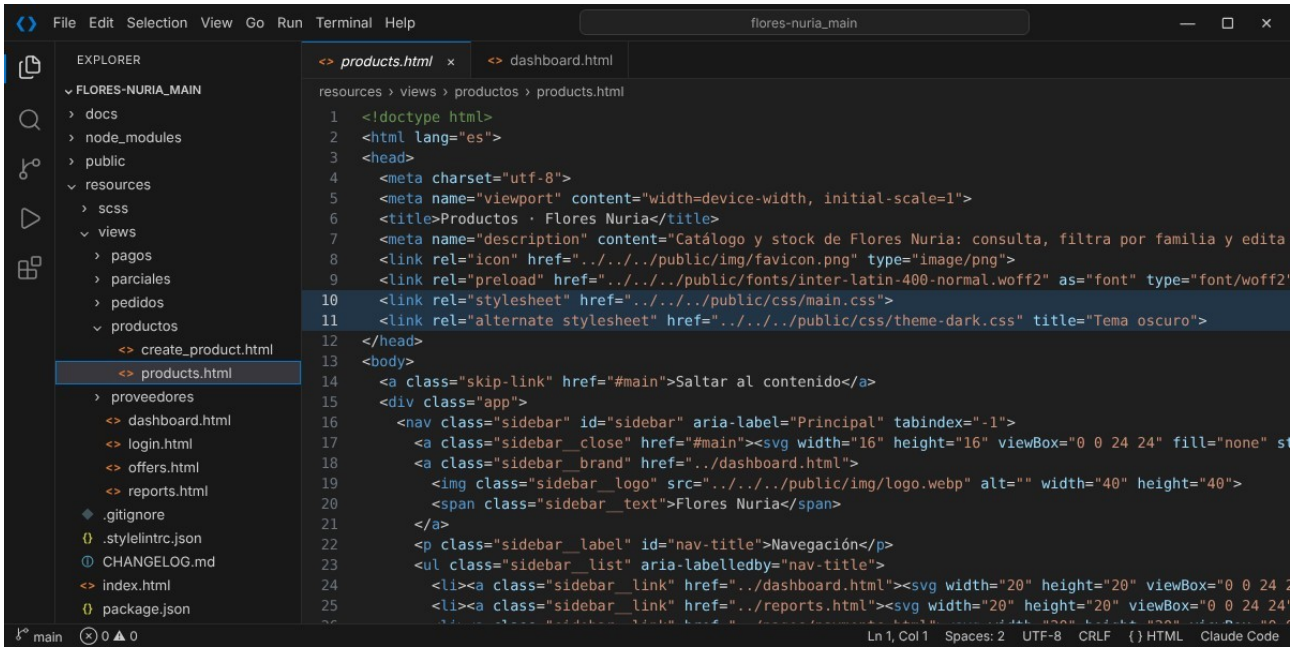


Figura 11. Cabecera de productos/products.html: la hoja principal (línea 10) y la alternativa (línea 11). En las subcarpetas la ruta empieza por ../../../ y en resources/views por ../../.

Comprobación automática: **19/19 vistas** enlazan `main.css` y `theme-dark.css`, todas las rutas existen y ninguna vista tiene estilos incrustados.

5.4. Compilación reproducible

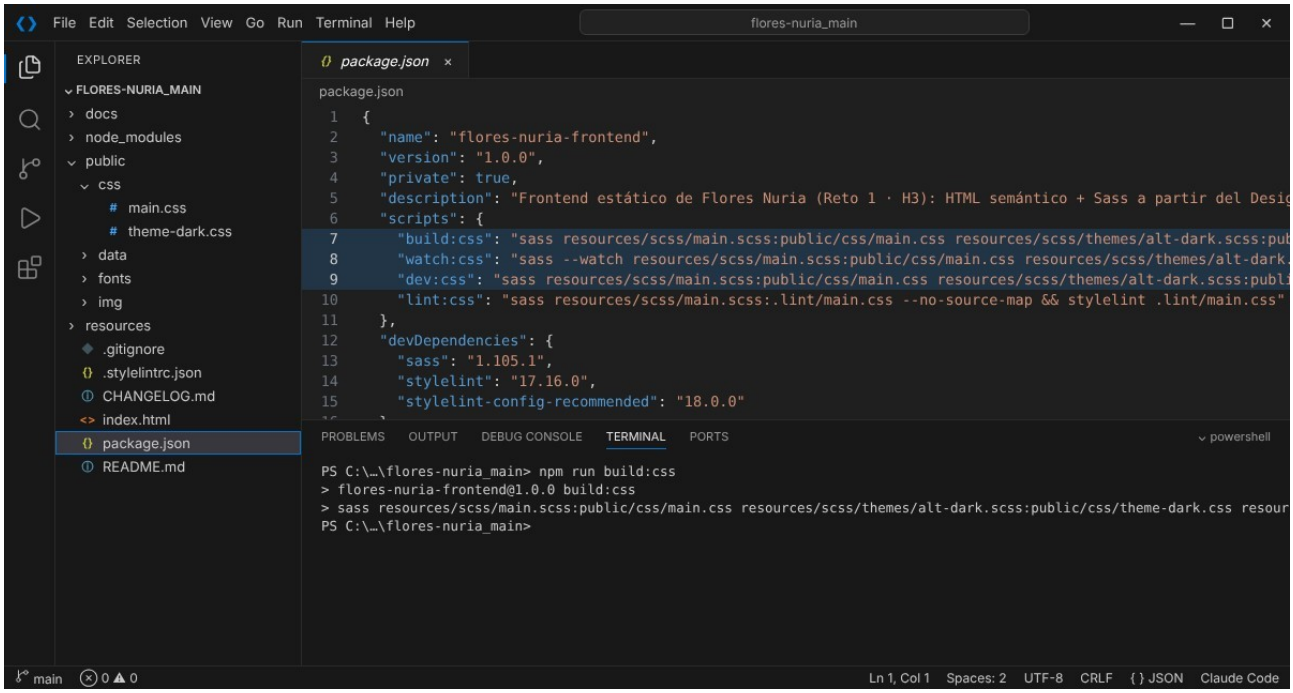
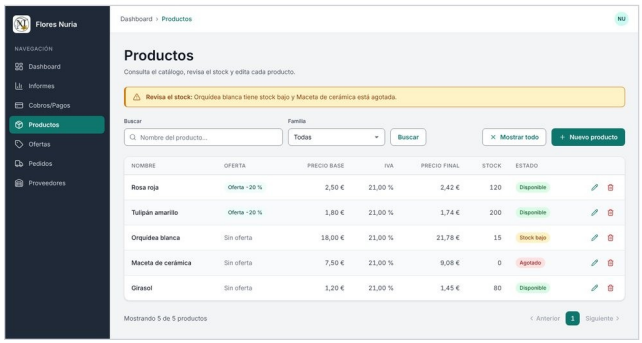


Figura 12. npm run build:css compila las cuatro hojas con una sola orden (scripts de package.json).

5.5. Mantenimiento: un cambio global sin tocar las vistas

Para demostrarlo cambiamos solo una línea de `_tokens.scss` (`primary-strong` y `sidebar-active` a violeta) y volvimos a compilar. Sin editar ningún HTML, cambian a la vez el botón primario, la opción activa del menú, los enlaces, la paginación y los iconos de acción en todas las vistas. Después restauramos el valor original.

primary-strong: #0F766E



primary-strong: #7C3AED (una línea cambiada)

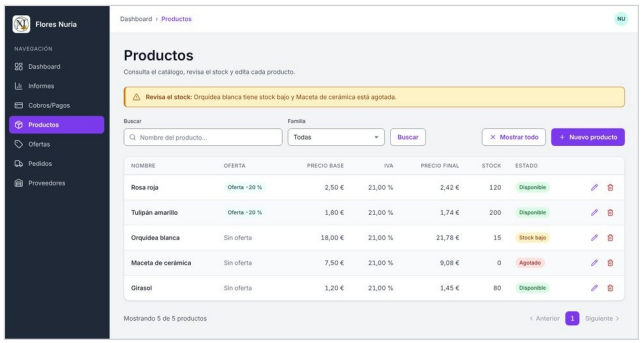
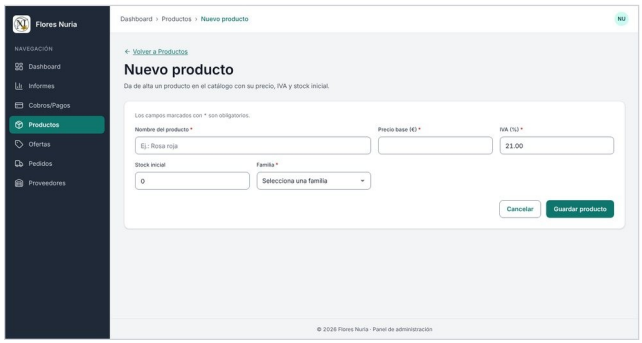


Figura 13. Productos antes y después de cambiar un solo token.

Nuevo producto · antes



Nuevo producto · después

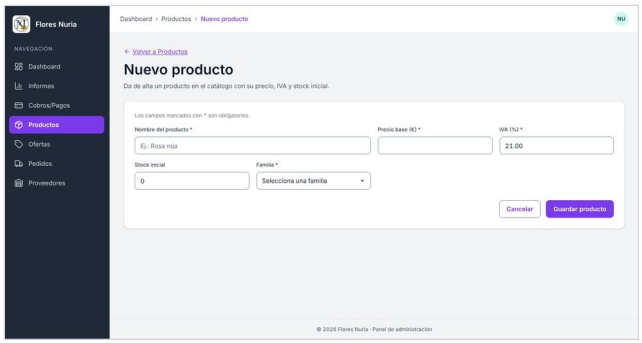


Figura 14. El mismo cambio llega al formulario «Nuevo producto» sin tocarlo.

6. CE2D · Hoja de estilos alternativa: tema oscuro

La variante visual del Hito 2 (tema oscuro) es una **hoja alternativa real**: `public/css/theme-dark.css`, que se enlaza en todas las vistas con `rel="alternate stylesheet"`. El navegador no la aplica hasta que se elige. Solo redefine tokens, así que **el HTML es exactamente el mismo**.

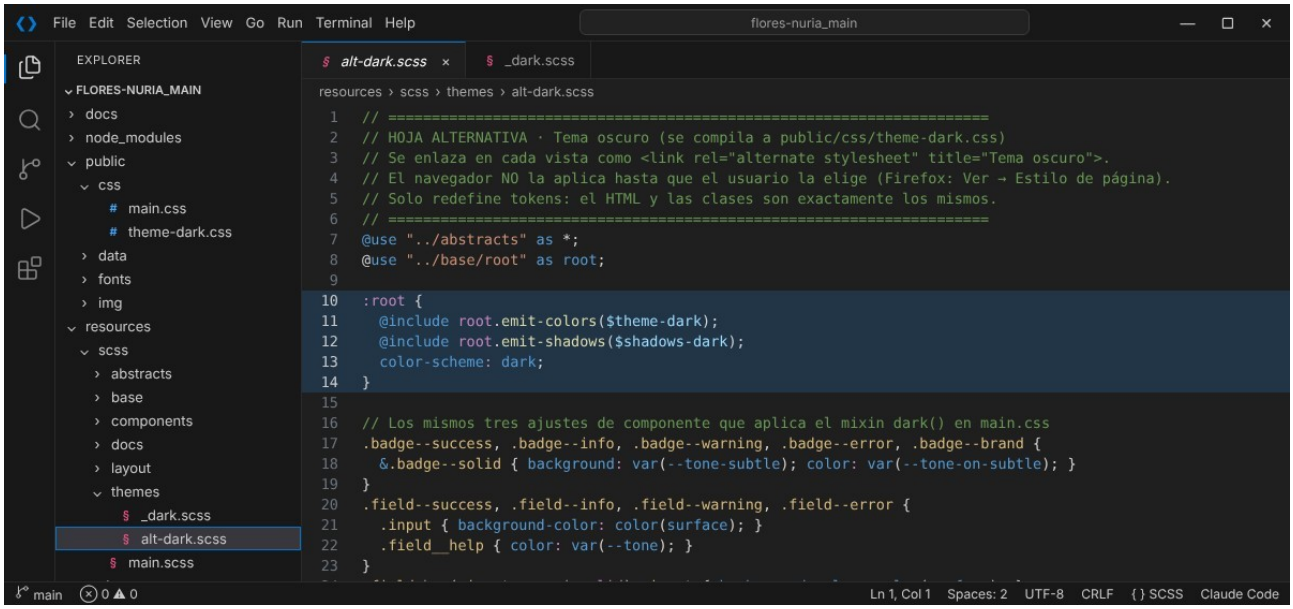


Figura 15. resources/scss/themes/alt-dark.scss: emite los 55 tokens del tema oscuro sobre :root y los tres ajustes de componente.

6.1. Cómo activar cada versión

Forma	Cómo se hace	Para qué
Hoja alternativa	Firefox: menú Ver → Estilo de página → «Tema oscuro» (Alt para ver el menú). «Estilo básico de página» vuelve al claro.	Elegir la variante a mano, como pide el CE2D
Preferencia del sistema	Windows: Configuración → Personalización → Colores → Modo oscuro. main.css lo detecta con `prefers-color-scheme`.	Que el usuario la tenga sin hacer nada
DevTools	Chrome: F12 → ⋮ → More tools → Rendering → «Emulate CSS media feature prefers-color-scheme: dark».	Comprobar las dos versiones en el mismo navegador
Atributo	Escribir `data-theme="dark"` (o `"light"`) en la etiqueta `<html>`.	Forzar un tema en una vista concreta al probar

6.2. Comprobación de las dos versiones

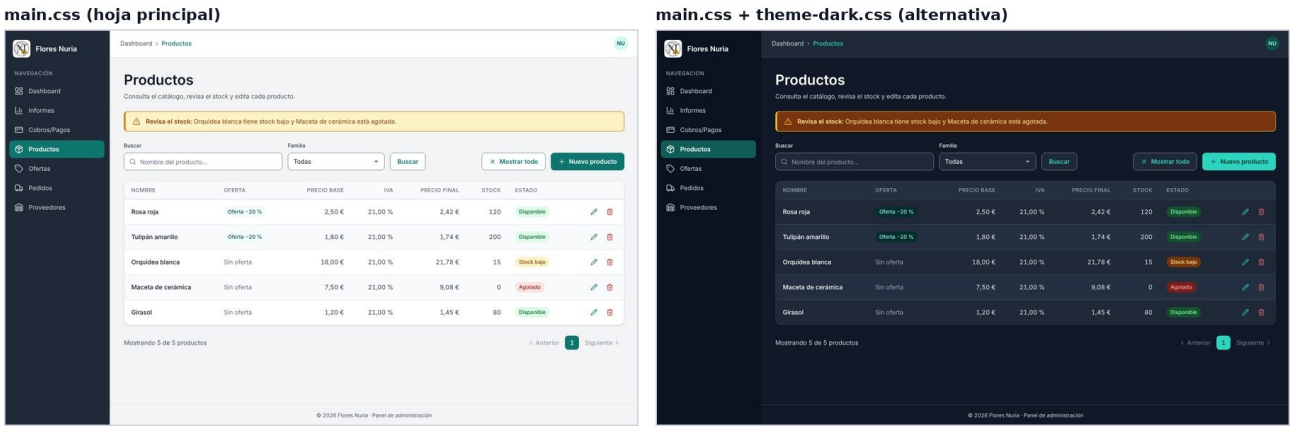


Figura 16. La misma vista con solo la hoja principal y con la hoja alternativa activada.

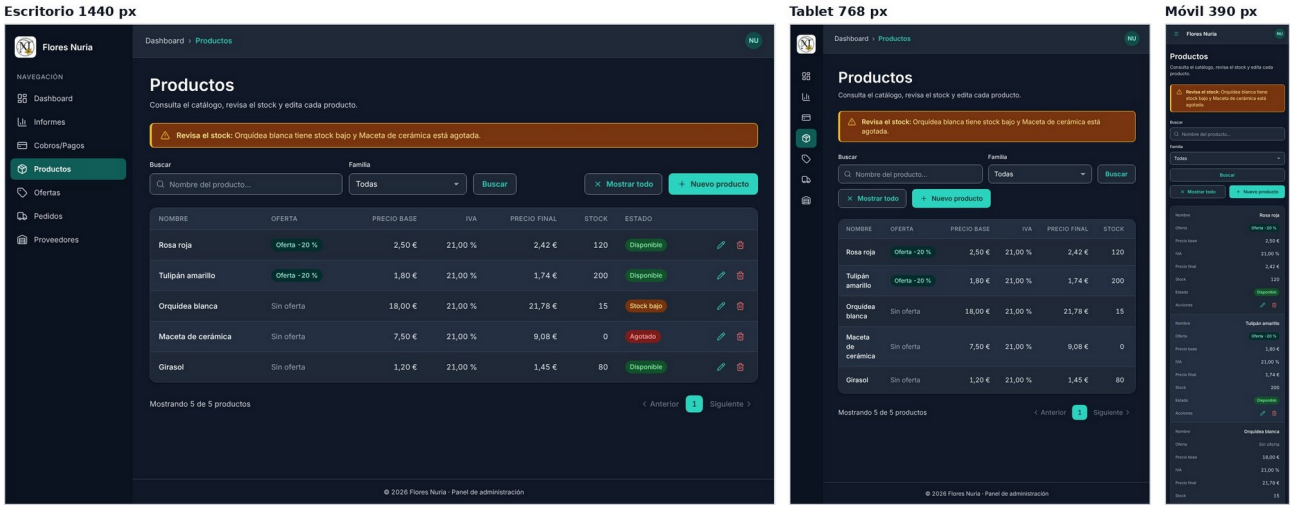


Figura 17. Hoja alternativa en Productos: escritorio, tablet y móvil.

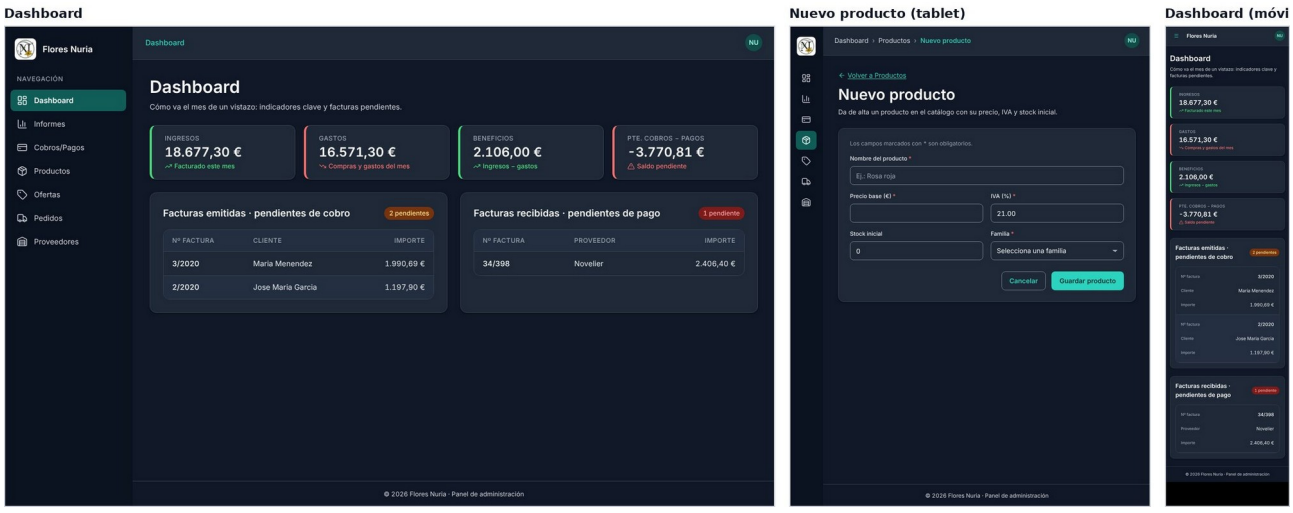


Figura 18. Hoja alternativa en el Dashboard y en un formulario.

Comprobación	Resultado
Accesibilidad automática (axe, WCAG 2.1 AA) con la hoja alternativa	0 problemas
Desbordamiento horizontal en 3 vistas × 3 tamaños con la hoja alternativa	0

Comprobación	Resultado
Contraste del tema oscuro (calculado en el Hito 2, apartado 4.4)	Todo el texto ≥ 4,5:1 y los bordes ≥ 3:1
Conflictos con la hoja principal	Ninguno: theme-dark.css solo redefine tokens y 3 reglas de componente
Cobertura	Todos los componentes y estados (botones, campos, badges, alertas, tablas, menús, gráficos) de las 19 vistas

7. CE2E · Cascada, herencia y redefinición

7.1. Depuración con las herramientas del navegador

El caso más claro lo encontramos en tablet: el menú lateral tenía que estrecharse a 72 px, pero seguía midiendo 240 px. Con **DevTools → Elements → Styles** vimos el motivo: la regla `@media … .sidebar { width: 72px }` aparecía tachada.

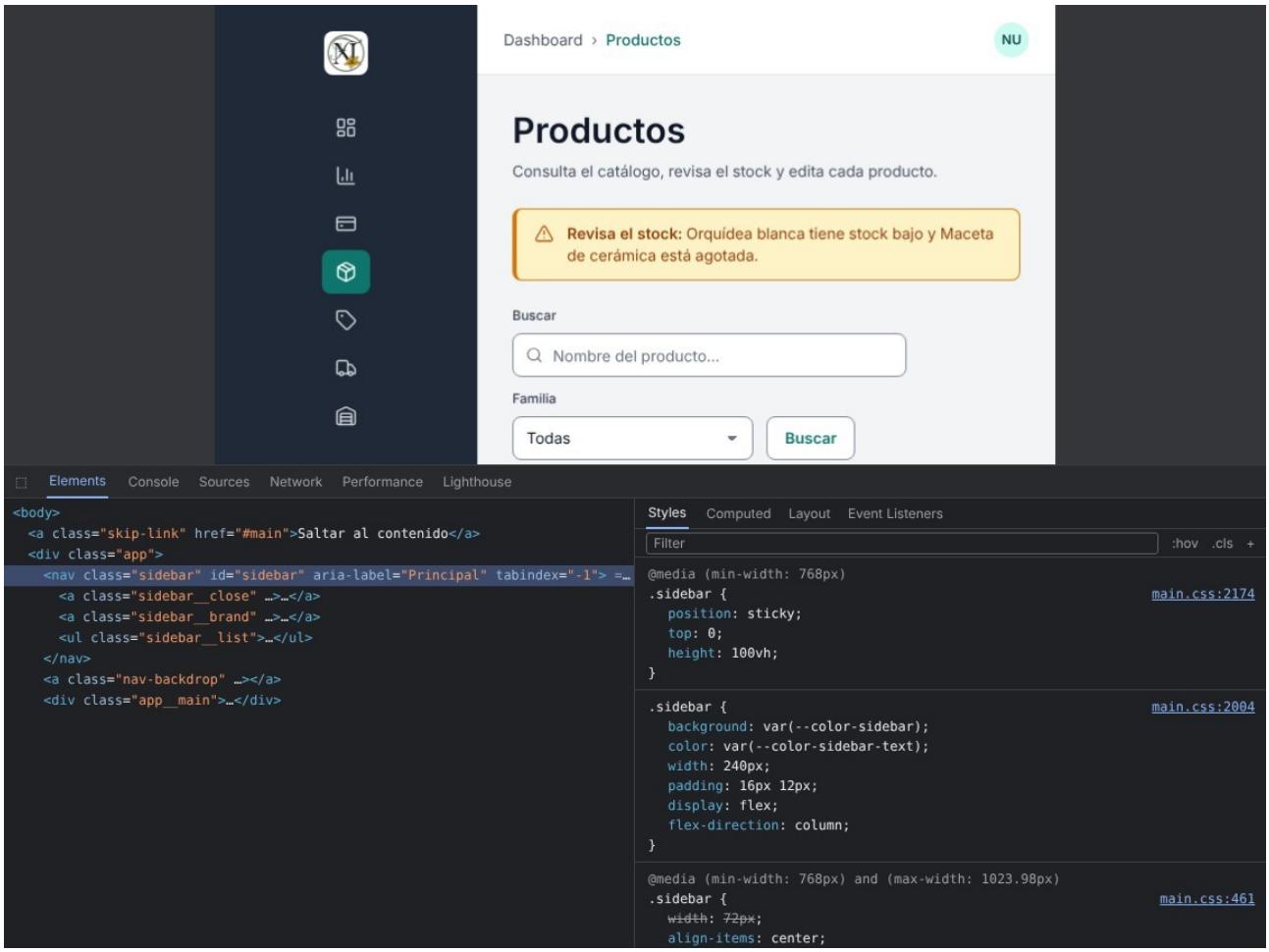


Figura 19. Antes: width: 72px (main.css:461) está tachada porque .sidebar { width: 240px } (main.css:2004) tiene la misma especificidad y aparece después.

Las dos reglas tienen la misma especificidad (0,1,0), así que gana la última. El responsive estaba en `layout/_app.scss`, que se compila antes que `components/_sidebar.scss`. La corrección fue mover el responsive del menú a su propio parcial, **después** de sus estilos base, sin subir la especificidad ni usar `!important`.

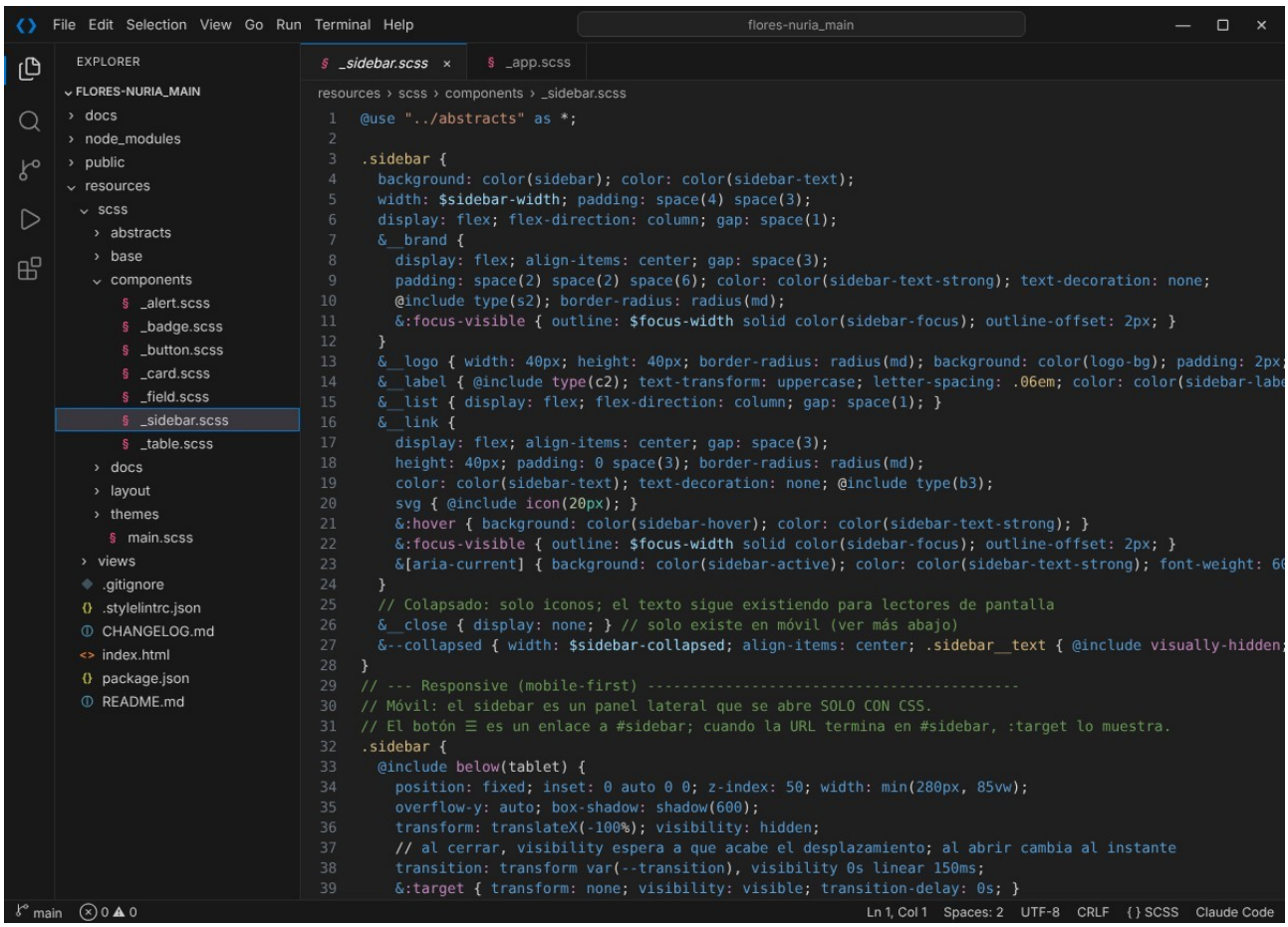


Figura 20. El responsive del menú (líneas 60–68) ahora va después de los estilos base dentro de _sidebar.scss.

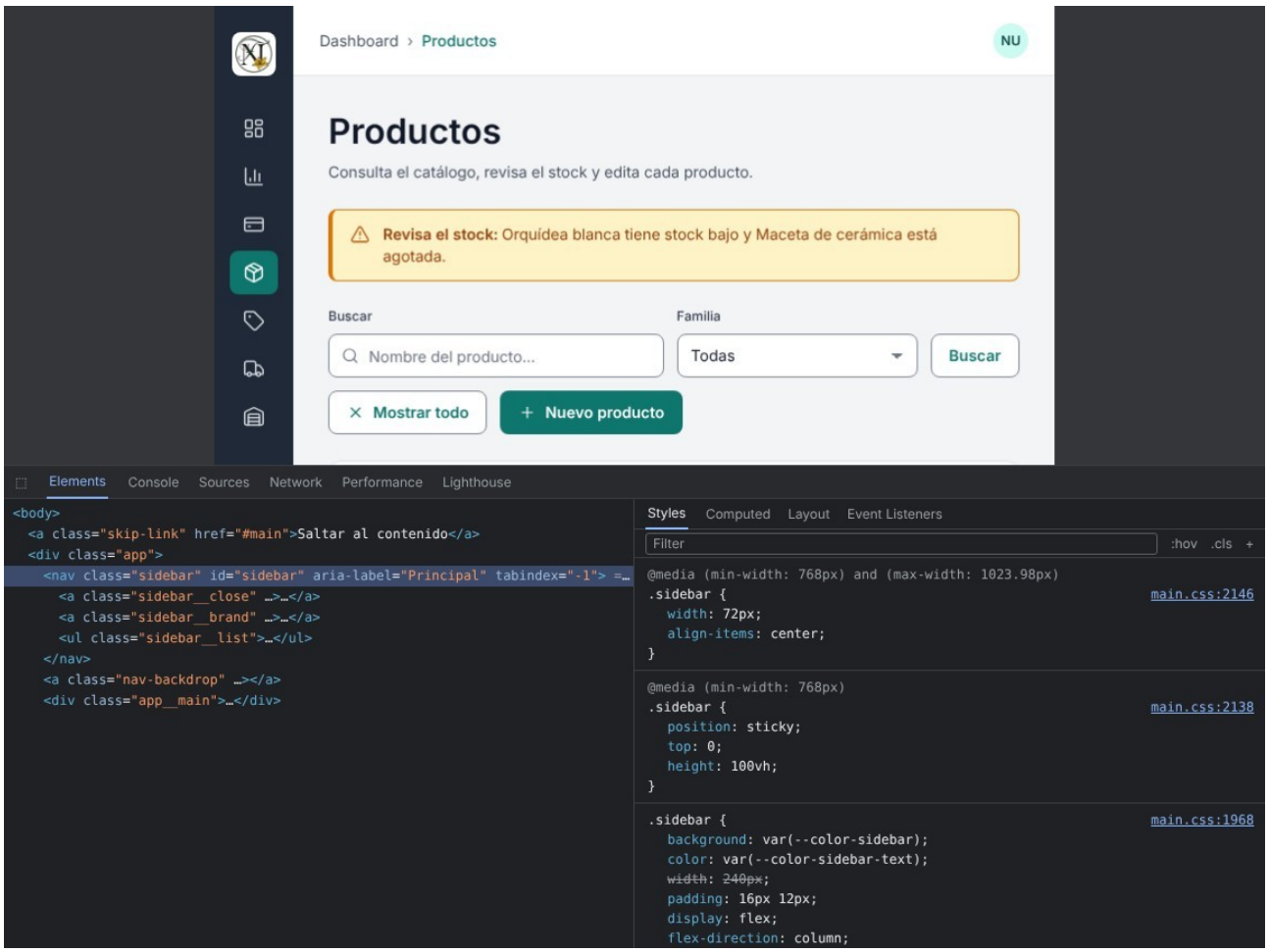


Figura 21. Después: la regla de tablet está arriba y es la que tacha width: 240px.

7.2. Otros casos de cascada y herencia

Caso	Qué regla prevalece y por qué
Orden LVHA en enlaces	`:visited` estaba después de `:hover` en el Hito 2, así que un enlace visitado no reaccionaba al ratón. Las cuatro pseudoclases tienen la misma especificidad: ahora van en orden link → visited → hover → active.
Herencia del body	El body fija fuente, tamaño, color y cifras tabulares; todo el texto lo hereda. Los títulos usan `color: inherit` y los controles `font: inherit` (por defecto no heredan la fuente).
Herencia en iconos	Los SVG usan `stroke="currentColor"`: heredan el color del botón o enlace, también en tema oscuro.
Inline frente a hoja	Ver apartado 4.1: el atributo style gana a la hoja salvo que la hoja use `!important`.

7.3. Redefinición controlada de variantes y estados

Las variantes no repiten el componente: **redefinen variables**. El mixin `tone()` cambia cuatro custom properties (`--tone`, `--tone-strong`, `--tone-subtle`, `--tone-on-subtle`) y el resto del componente las usa. Así `.badge--warning`, `.alert--error` o `.field--success` cambian sin efectos secundarios en el resto.

resources/scss/abstracts/_mixins.scss

```
@mixin tone($name) {
  --tone: var(--color-#{$name});
  --tone-strong: var(--color-#{$name}-strong);
  --tone-subtle: var(--color-#{$name}-subtle);
  --tone-on-subtle: var(--color-#{$name}-on-subtle);
}
```

El tema oscuro es la misma idea a gran escala: no tiene reglas por componente, solo redefine los tokens en `[data-theme="dark"]`, en `@media (prefers-color-scheme: dark)` y en la hoja alternativa.

7.4. Selectores y especificidad sin !important

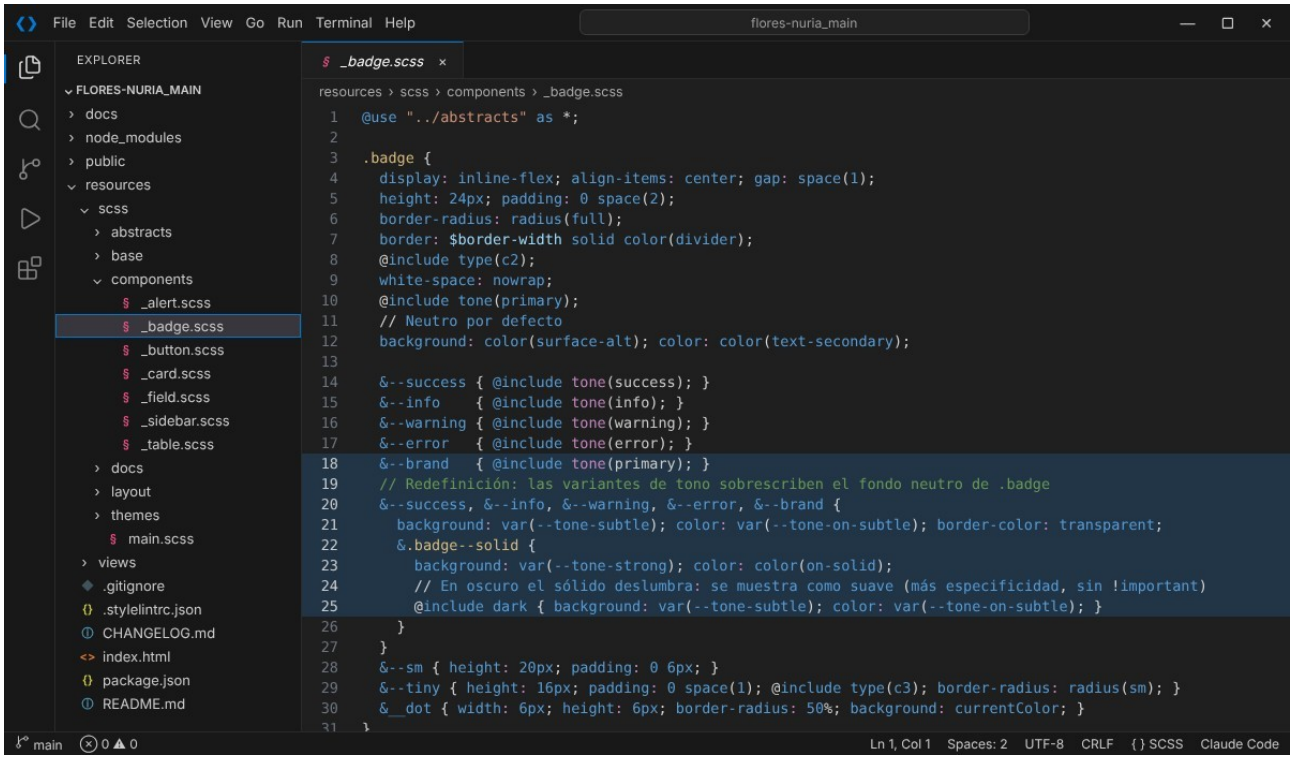


Figura 22. El badge sólido en oscuro gana por especificidad: [data-theme=dark] .badge--error.badge--solid (0,3,0) supera a .badge--error.badge--solid (0,2,0).

En el Hito 2 este caso y el estado vacío de la tabla usaban `!important`. Ahora ganan por especificidad (por ejemplo, `.table td.table__empty` (0,2,1) frente a `.table td` (0,1,1)). En la aplicación solo quedan dos `!important`, ambos justificados: `[hidden]` (un elemento oculto nunca debe mostrarse) y `prefers-reduced-motion` (manda la preferencia del usuario).

8. CE2F · Propiedades CSS

Categoría	Propiedades	Ejemplo real
Tipografía	font-family, font-size en rem, line-height, font-weight, letter-spacing, text-transform, font-variant-numeric, text-underline-offset	Mixin `type(b4, 600)`; cabeceras de tabla C2 en mayúsculas; cifras tabulares en importes
Espacios	padding, margin, gap con `space()` (múltiplos de 4 px)	`.card` padding space(6) → space(4) en móvil
Dimensiones	width/height de controles (40/32 px), min-width: 0, max-width, `min()`	`.modal` width: min(440px, 100% − 32px)
Distribución	display: grid / flex, grid-template-columns, minmax(0,1fr), flex-wrap, position: sticky / fixed	Esqueleto, KPI 1→2→4 columnas, barra superior fija
Estados	:hover, :focus-visible, :active, :visited, :disabled, :checked, :target, :user-invalid, :has(), [open], [aria-current], [aria-pressed]	Botón (6 estados), error de campo, menú móvil con :target
Color y efectos	background-color, border-color, box-shadow con shadow(), accent-color, transform, transition	Interruptor con transform; 8 niveles de sombra

8.1. Distribución con Grid y Flexbox

Esqueleto de la aplicación (layout/_app.scss) y rejilla de KPI

```
@include respond(tablet) {
  .app { display: grid; grid-template-columns: auto minmax(0, 1fr); }
}                        // auto = ancho del menú; minmax(0,1fr) evita que una tabla ancha
estire la página
.kpi-grid { display: grid; gap: space(4); grid-template-columns: 1fr;
  @include respond(tablet)  { grid-template-columns: repeat(2, 1fr); }
  @include respond(desktop) { grid-template-columns: repeat(4, 1fr); } }
.toolbar { display: flex; flex-wrap: wrap; gap: space(3); align-items: flex-end; }
```

Usamos **Grid** cuando hay filas y columnas a la vez (esqueleto, KPI, formularios de 1/2/4 columnas) y **Flexbox** para alinear en una dirección (barra superior, barra de acciones, botones con icono). `min-width: 0` en los hijos flex y `minmax(0, 1fr)` en Grid impiden que el contenido largo desborde.

8.2. Combinaciones de propiedades explicadas

Combinación	Efecto
`position: sticky; top: 0; z-index: 20`	La barra superior se queda fija al hacer scroll sin sacarla del flujo.
`appearance: none` + dos `linear-gradient` + `background-position`	El select pierde la flecha del sistema y dibujamos una con el color del token (se adapta al tema).
`position: absolute; width: 1px; height: 1px; clip-path: inset(50%)`	Texto solo para lectores de pantalla (.visually-hidden), sin el `clip` obsoleto.
`transform: translateX(-100%)` + `visibility: hidden` + `transition` con retardo	El menú móvil sale del lateral y no recibe el foco mientras está cerrado.
`.field:has(.input:user-invalid)`	El campo entero se marca en rojo y muestra su mensaje solo después de que el usuario lo toque o intente enviar.

8.3. Aplicación del diseño del Hito 2

Decisión del Hito 2	En el frontend
Teal #0F766E para el botón primario y los enlaces (5,47:1)	Token primary-strong en .btn--primary, .link, paginación y menú
Grises según el fondo (#6B7280 solo sobre blanco)	text-muted sobre el fondo gris; la guía detectó y corrigió un caso (apartado 11)
Inter 400/500/600 autoalojada	public/fonts, subconjunto latino de 48 KB
Escala tipográfica con versión móvil	type-responsive(h3): 32 px → 24 px por debajo de 768 px
Plantilla de listado y alternativas elegidas	Cobros, Productos, Ofertas, Pedidos y Proveedores

9. CE2G · Clases CSS

9.1. Convención de nombres

Usamos **BEM**: `.bloque` (componente), `.bloque__elemento` (parte) y `.bloque--modificador` (variante). Por ejemplo: `.kpi`, `.kpi__value`, `.kpi--positive`. Los estados no son clases: se expresan con pseudoclases o atributos ARIA (`[aria-current]`, `[aria-pressed]`, `:disabled`), así el estado visual y el accesible son siempre el mismo.

9.2. Reutilización en las vistas

Componente	Vistas	Usos
Sidebar + topbar + migas	18	18
Cabecera de página	18	18
Tarjeta .card	16	37
Campo .field / .input	10	51
Botón primario / secundario	11 / 9	16 / 23
Tabla .table	8	10
Rejilla .form-grid	8	9
Badge	5	20
Paginación	5	5
Pop-up de confirmación	4	4
Tarjeta KPI	2	8
Estado vacío	6	6

9.3. Variantes y estados

Un mixin genera todas las variantes del botón (components/_button.scss)

```
&--primary      { @include button-variant(color(primary-strong), color(on-primary),
color(primary-hover), color(primary-active)); }
&--secondary    { @include button-variant(color(surface), color(primary-strong),
color(primary-subtle), color(primary-muted), color(border-control)); }
&--destructive  { @include button-variant(color(error-strong), color(on-error),
color(error-hover), color(error-active)); }
&--ghost-danger { @include button-variant(transparent, color(error-strong), color(error-
subtle), color(error-subtle)); }
&--sm { … }  &--lg { … }  &--block { … }  &--icon { … }
&:disabled, &[aria-disabled="true"] { … }   &[aria-busy="true"] { cursor: progress; }
```

La tabla completa de clases, variantes y estados está en la guía de estilo (apartado 11). **Mantenibilidad:** como cada variante solo cambia variables, un cambio en `.btn` o en un token se propaga a todas las vistas sin efectos imprevistos (demostrado en el apartado 5.5).

10. CE2H · Validación CSS

Usamos dos herramientas complementarias:

- **W3C CSS Validator** (el oficial, la misma versión que jigsaw.w3.org, ejecutado en local): comprueba que la sintaxis y los valores sean CSS válido.
- **Stylelint** con la configuración recomendada: detecta errores que el validador no ve, como propiedades duplicadas o selectores repetidos. Lo hemos dejado configurado en el proyecto (`npm run lint:css`).

Validamos la versión **expandida** del CSS (`npm run dev:css`) para que el informe dé números de línea útiles; la versión comprimida es el mismo CSS.

10.1. Resultados antes y después

Versión	W3C · errores	W3C · advertencias	Stylelint · errores	Stylelint · advertencias
main.css del Hito 2	0	9	5	6
Primera versión del H3 (la del repositorio)	0	19	9	14
Versión final	**0**	**19**	**0**	**14**



Figura 23. W3C CSS Validator sobre el main.css del Hito 2: 0 errores y 9 advertencias (dos clip obsoletos).

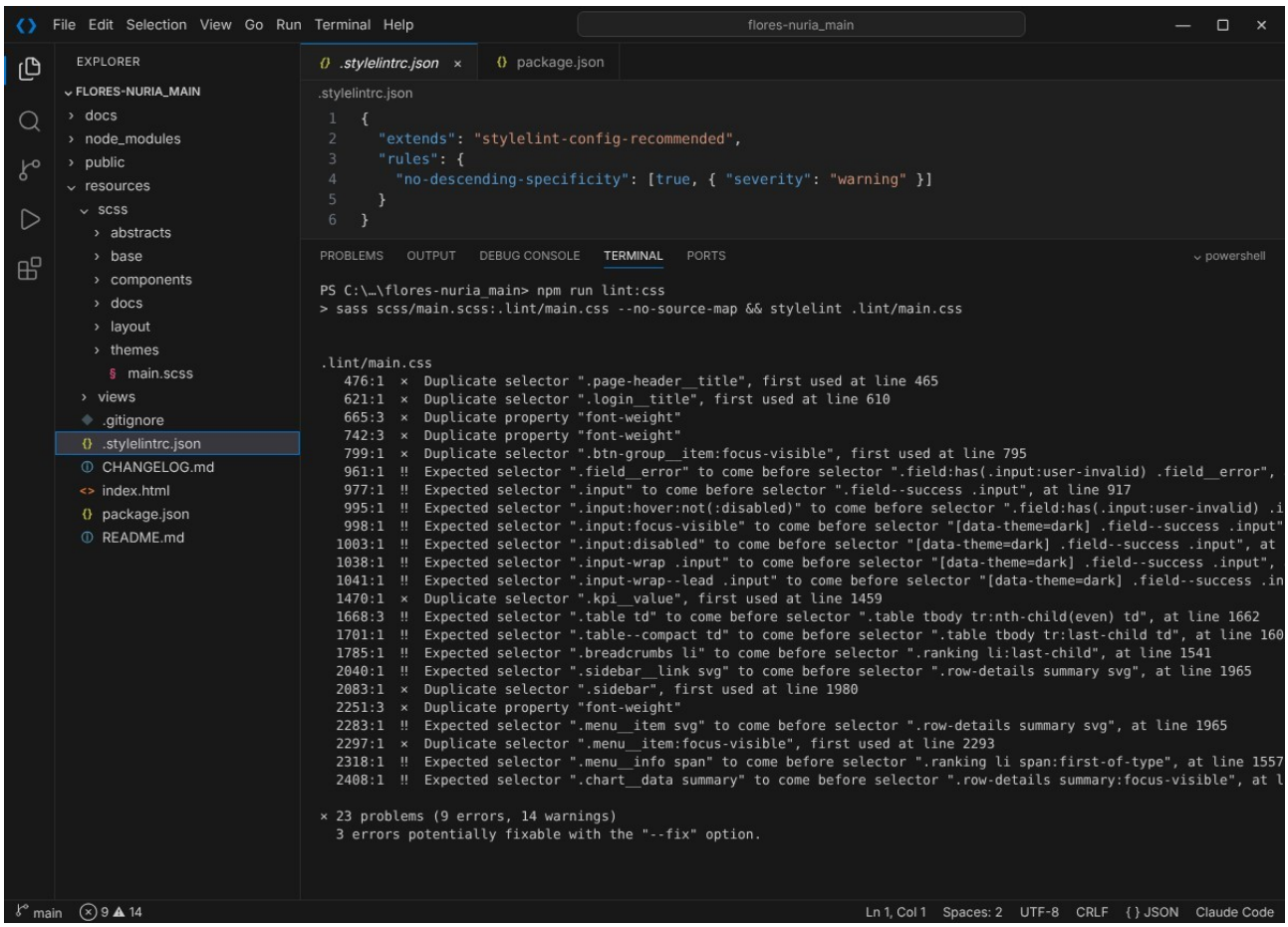


Figura 24. Stylelint sobre la primera versión del H3: 23 problemas (9 errores y 14 advertencias).

10.2. Errores detectados y corregidos

Error real	Dónde	Corrección
`font-weight` duplicado en la misma regla (3)	.btn, .btn--sm, .avatar	El mixin `type()` acepta el peso como 2.º parámetro: `@include type(b4, 600)`.
Selector duplicado por un @media en mitad de la regla (3)	.page-header__title, .login__title, .kpi__value	La declaración de color va antes del `@include type-responsive()`, así Sass genera una sola regla.
`:focus-visible` declarado dos veces (2)	.btn-group__item, .menu__item	El mixin `focus-ring($offset)` recibe el desplazamiento en lugar de sobrescribirlo después.
Selector `.sidebar` repetido (1)	components/_sidebar.scss	Se unen los dos bloques; el responsive queda dentro del parcial.
Propiedad `clip` obsoleta (2, en el Hito 2)	.visually-hidden	Sustituida por `clip-path: inset(50%)`.
Opacidad del botón «cargando» (detectado con axe)	.btn[aria-busy]	Se quita `opacity: .85`: bajaba el contraste del texto a 4,06:1.

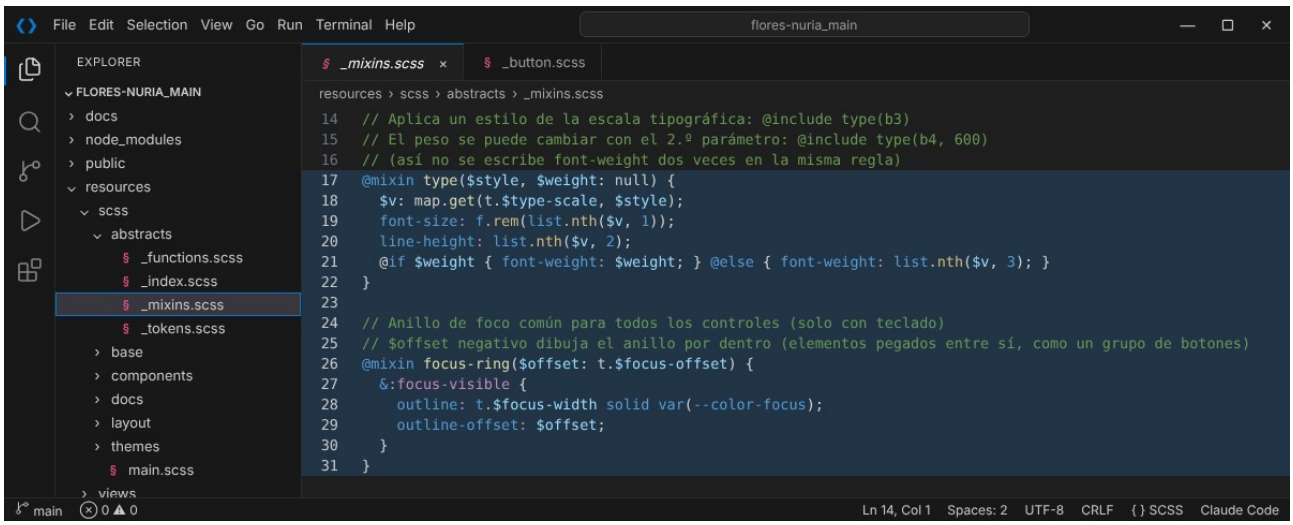


Figura 25. Corrección en _mixins.scss: type($style, $weight) y focus-ring($offset).

10.3. Revalidación y advertencias que se mantienen

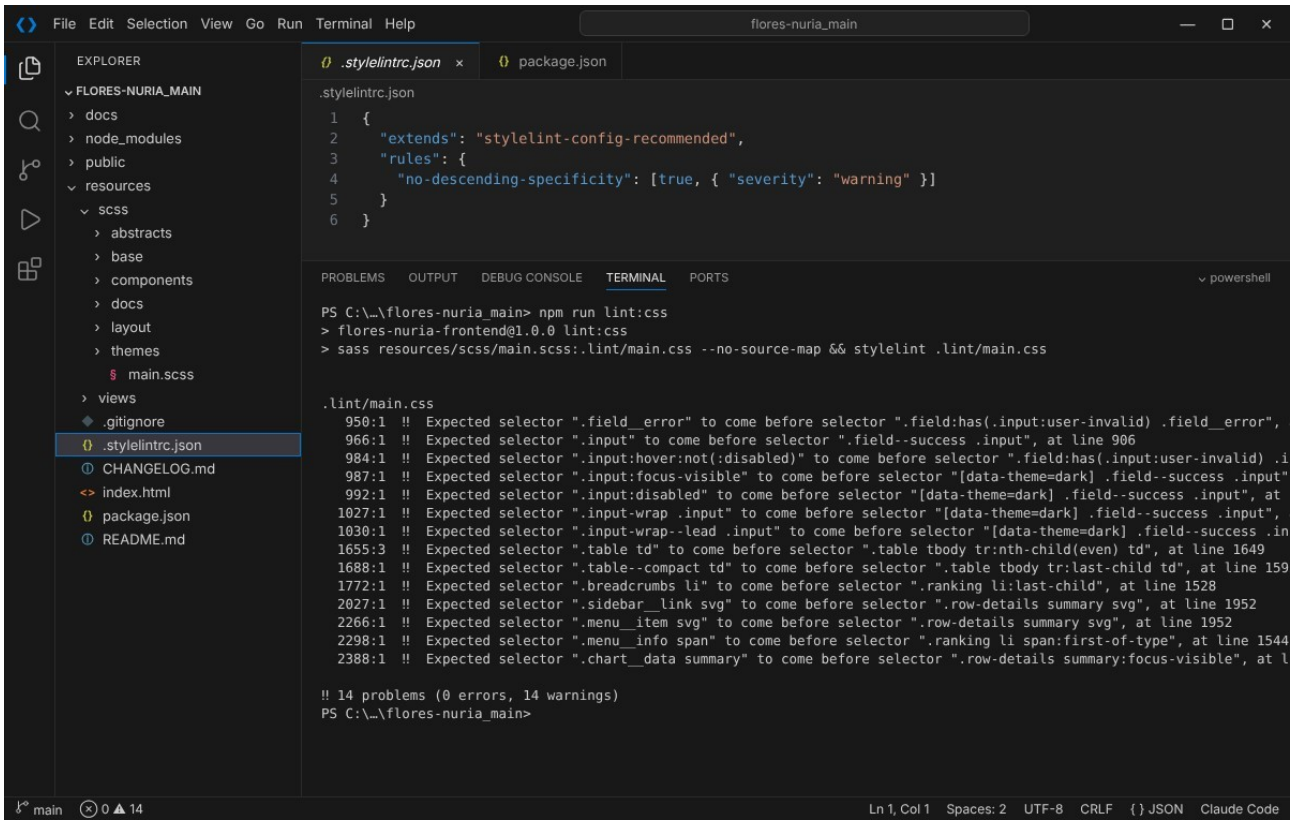


Figura 26. Revalidación con Stylelint: 0 errores y 14 advertencias.

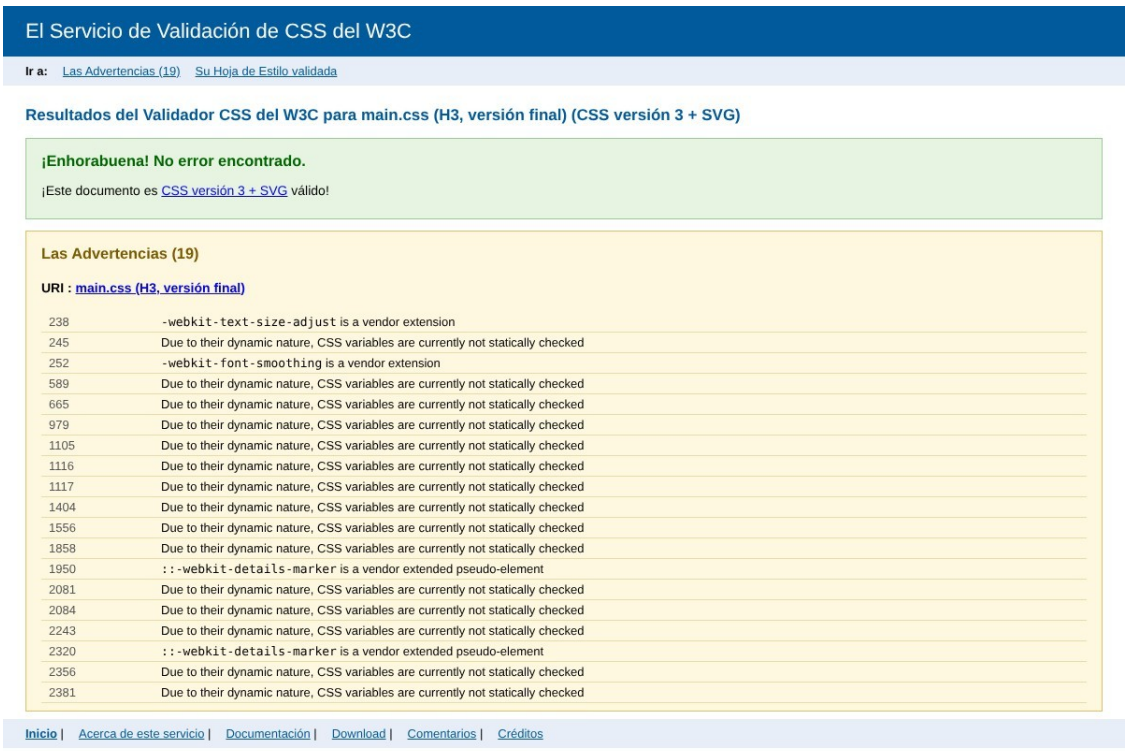


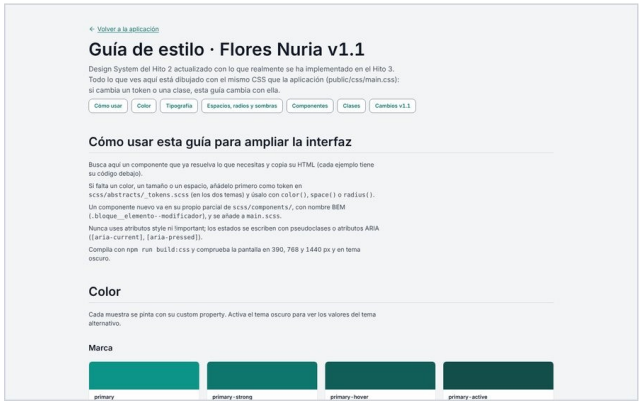
Figura 27. W3C CSS Validator sobre la versión final: 0 errores.

Advertencia	Herramienta	Por qué se mantiene
«Las variables CSS no se comprueban de forma estática» (15)	W3C	Es informativa: el validador no puede saber el valor de var(--…), que depende del tema.
-webkit-text-size-adjust · -webkit-font-smoothing · ::-webkit-details-marker ×2	W3C	Prefijos necesarios para Safari/macOS; no afectan al resto de navegadores.
no-descending-specificity (14)	Stylelint	Avisa cuando un selector menos específico va después de uno más específico, aunque afecten a elementos distintos (p. ej. `.menu__item svg` y `.row-details summary svg`). Revisados uno a uno: ninguno provoca un conflicto real, por eso lo configuramos como advertencia en .stylelintrc.json.

11. CE2I · Guía de estilo actualizada

Hemos actualizado el Design System del Hito 2 a la **versión 1.1** y lo hemos publicado como una página del propio proyecto: `docs/guia-estilo.html`. Está hecha con el mismo `main.css` que la aplicación, así que **muestra exactamente lo implementado**: si cambia un token o una clase, la guía cambia con ella. Las muestras de color se generan desde los mapas de tokens.

Guía de estilo · claro



Guía de estilo · oscuro

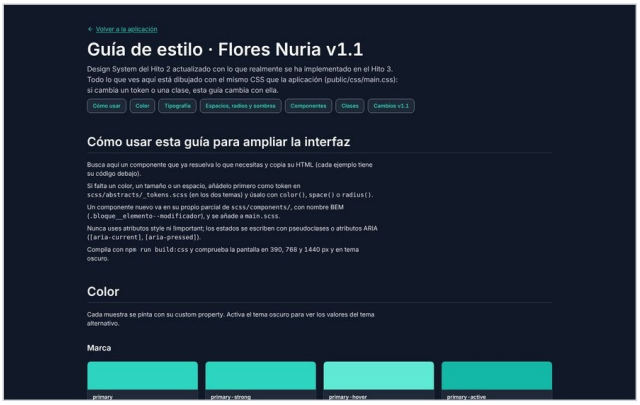


Figura 28. Guía de estilo v1.1 en tema claro y oscuro.

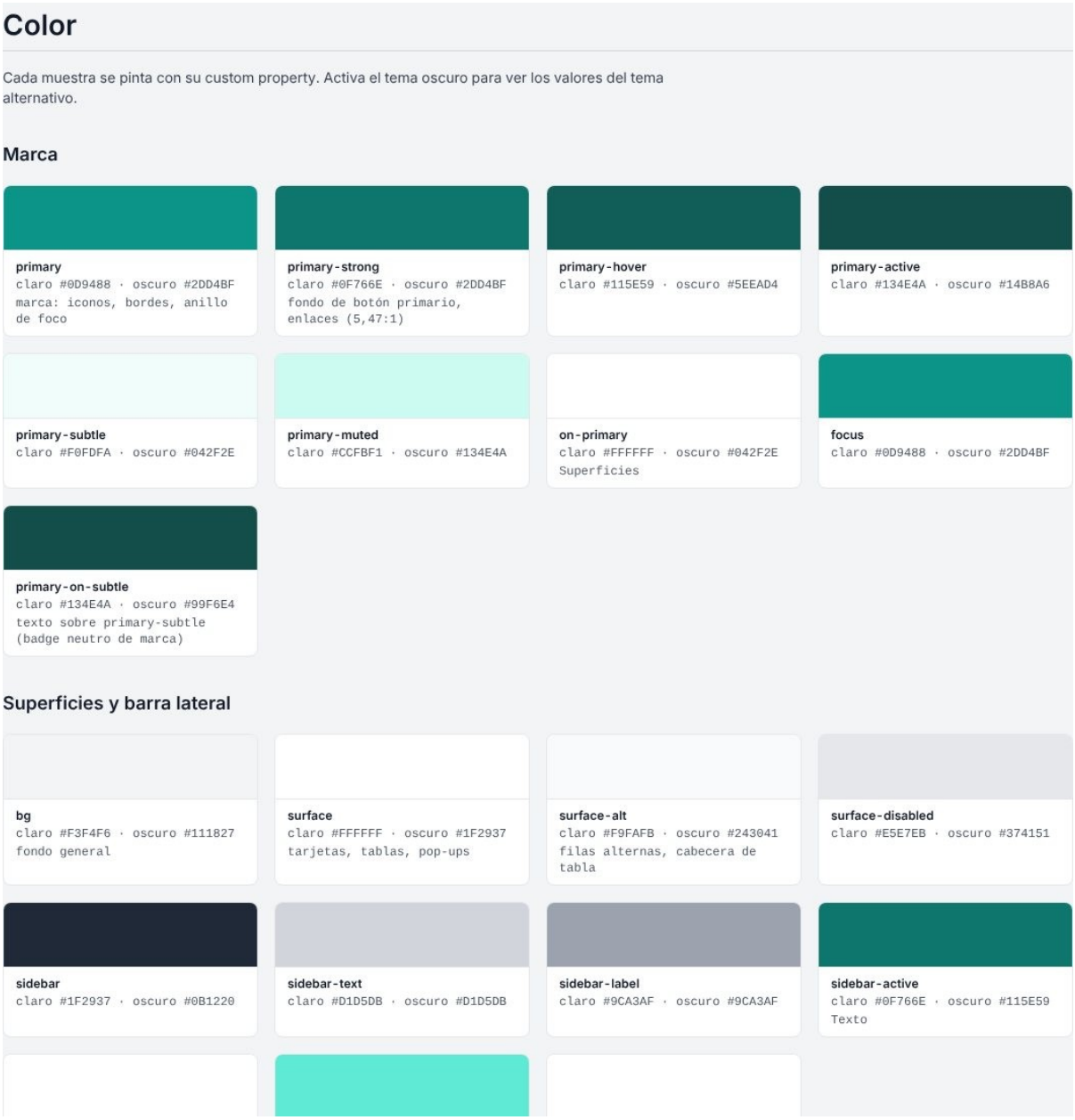


Figura 29. Colores con su token, su valor en cada tema y su uso.

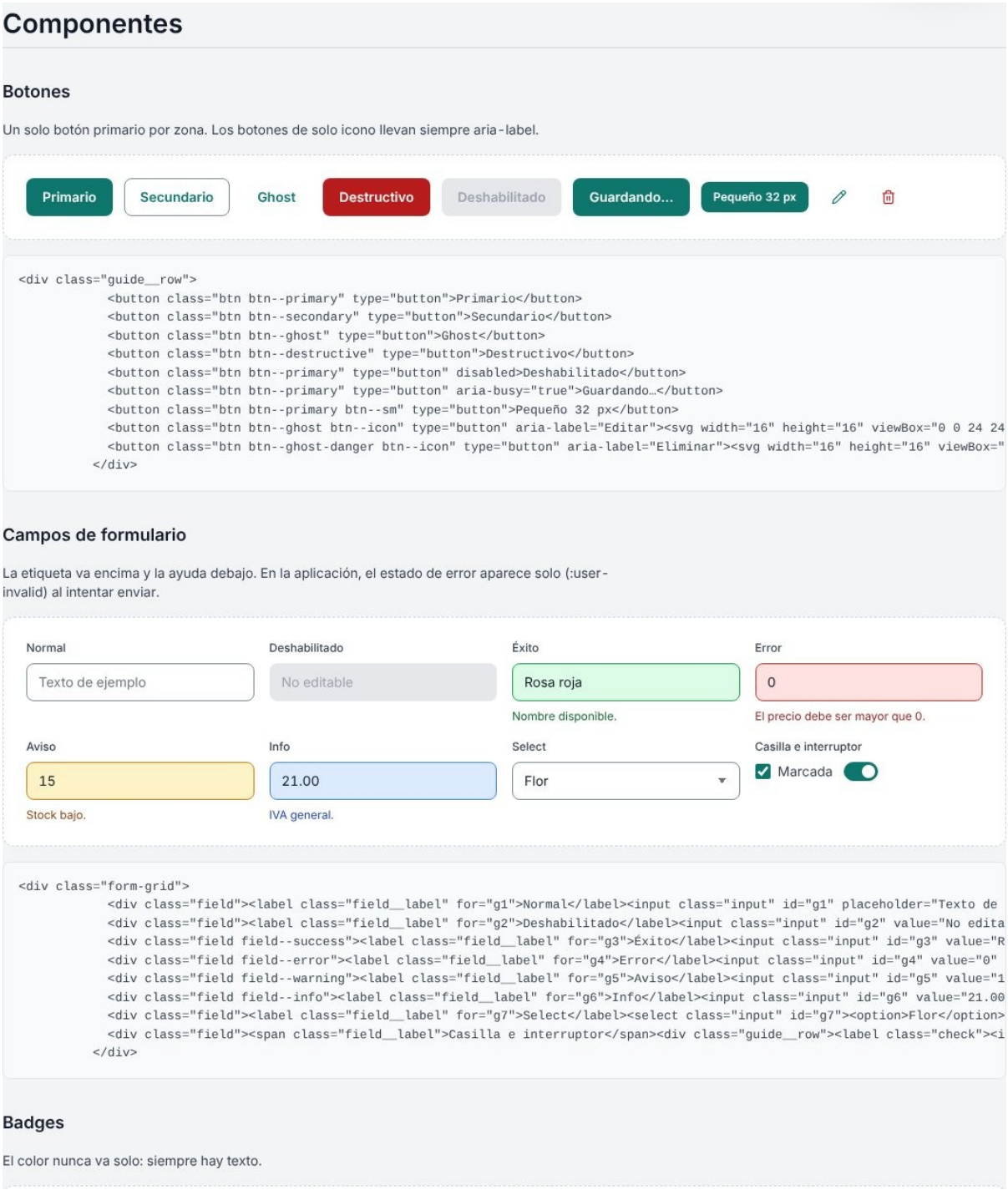


Figura 30. Componentes con sus variantes y estados, y el HTML que hay que copiar debajo de cada uno.

11.1. Cambios respecto al Hito 2

Elemento	Cambio	Motivo
Tokens	13 tokens nuevos en los dos temas (link-visited, on-error, error-hover, sidebar-focus, logo-bg, chart-1…)	Sustituyen colores escritos a mano y colorean gráficos y estados con el tema
Tema oscuro	Antes: botón sol/luna. Ahora: hoja alternativa theme-dark.css + preferencia del sistema + data-theme	Frontend solo HTML y CSS; la hoja alternativa es el mecanismo estándar
Pop-up	Antes: foco atrapado y Esc. Ahora: se abre con :target y se cierra con «Cancelar»	Sin código; mantiene el aviso de acción irreversible

Elemento	Cambio	Motivo
Menús	Menú móvil con :target y «Cerrar menú»; menú de usuario con <details>	Funcionan con teclado y sin scripts
Formularios	Error automático con :user-invalid y .field__error	El error aparece al enviar, no al cargar
Componentes nuevos	Grupo de botones, interruptor, lista de casillas, detalle de fila, gráfico, estado vacío, «Saltar al contenido»	Pantallas reales que no estaban en el Hito 2
Botón «cargando»	Sin opacidad	El 85 % de opacidad bajaba el contraste a 4,06:1
Mixins	type($estilo, $peso) y focus-ring($offset)	Errores detectados al validar



Figura 31. La misma tabla de cambios dentro de la guía.

11.2. Cómo usar la guía para ampliar la interfaz

5. Buscar un componente que ya resuelva el caso y copiar su HTML de la guía.
6. Si falta un color o un espacio, añadirlo como token en `_tokens.scss` (en los dos temas) y usarlo con `color()`, `space()` o `radius()`.
7. Un componente nuevo va en su propio parcial con nombre BEM y se añade a `main.scss`.
8. No usar atributos style ni `!important`; los estados, con pseudoclases o ARIA.
9. Compilar, pasar `npm run lint:css` y revisar 390, 768 y 1440 px en los dos temas.

12. Diseño responsive

Tamaño	Cambios principales
Móvil · 390 px	Menú lateral oculto que se abre con ☰ (:target); tablas como tarjetas apiladas; formularios a 1 columna; botones a todo el ancho; títulos H3 a 24 px
Tablet · 768 px	Menú colapsado a 72 px (solo iconos; el texto sigue para lectores de pantalla); 2 KPI por fila; formularios a 2 columnas
Escritorio · 1440 px	Menú de 240 px; 4 KPI por fila; Dashboard e Informes en dos columnas; formularios a 4 columnas

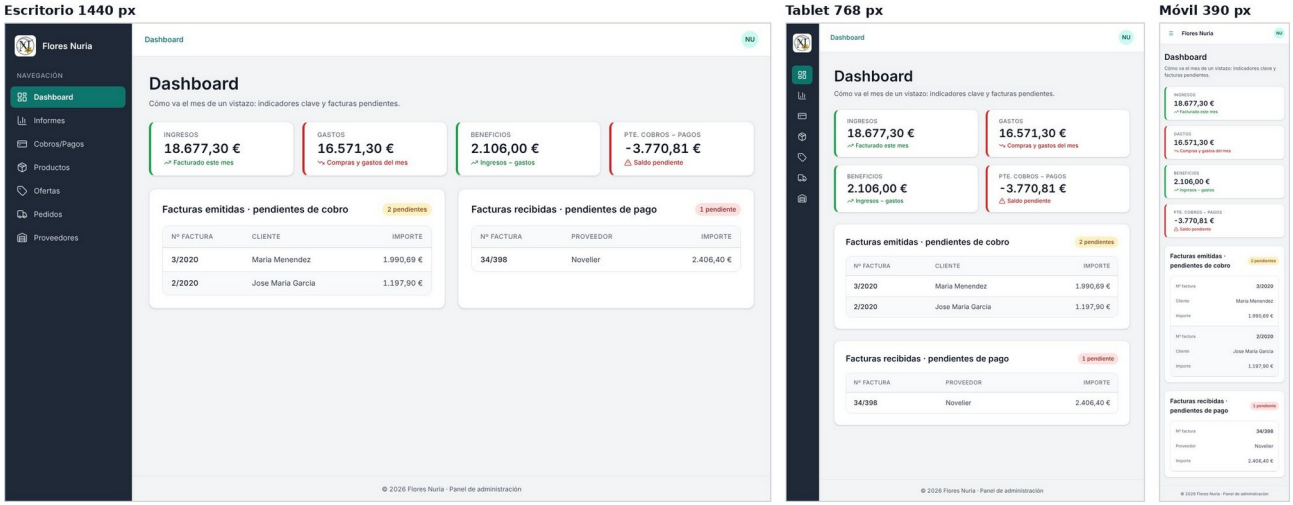


Figura 32. Dashboard en escritorio, tablet y móvil.

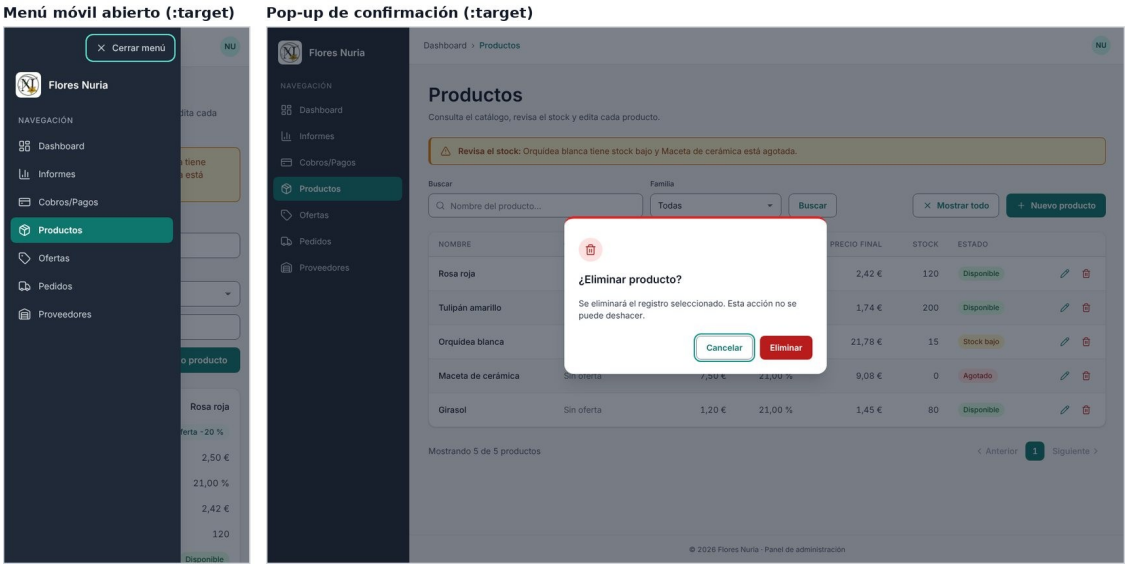


Figura 33. Menú móvil abierto y pop-up de confirmación, los dos con :target.

Comprobación automática: 14 vistas × 3 tamaños × 2 temas sin desbordamiento horizontal.

13. Validación HTML y accesibilidad

Herramienta	Antes	Después
Nu Html Checker (W3C) · 19 vistas	26 errores · 219 avisos	0 errores · 0 avisos
axe-core (WCAG 2.1 A/AA) · 14 vistas	79 elementos con problemas (7 críticos)	0
axe-core · guía y práctica	—	0 (tras corregir el contraste del botón «cargando» y de las etiquetas grises)
Teclado	Menú móvil inutilizable	6 tareas completas solo con teclado; 0 bloqueos

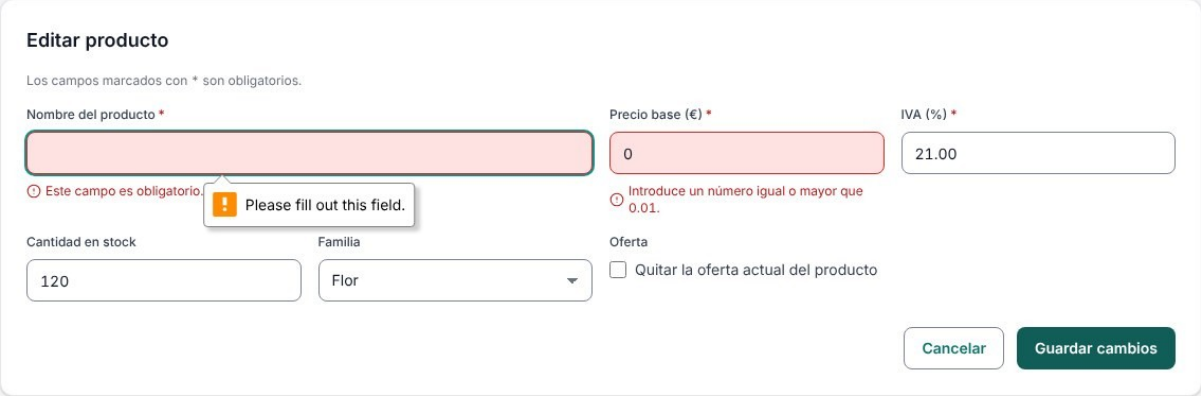


Figura 34. Errores de formulario con :user-invalid tras intentar guardar.

14. Ecodiseño: mediciones antes y después

Métrica	Antes	Después	Variación
Imágenes	47,1 KB (3)	8,1 KB (2)	−83 %
Colores escritos a mano en estilos de componentes	145	0	−100 %
Estilos en línea + colores en atributos SVG	3 + 25	0 + 0	−100 %
Clases de utilidad distintas en el HTML	44	1	−98 %
CSS principal (gzip) · hoja alternativa (gzip)	4,4 KB · —	7,1 KB · 1,4 KB	+93 %
Fuentes	0 (sistema)	47,5 KB	nuevo
Primera visita a Productos	42,4 KB	68,2 KB	+61 %
Visitas siguientes (solo HTML)	1,9 KB	4,0 KB	+2,0 KB

Las imágenes bajan un 83 % y desaparecen los valores sueltos. La primera visita pesa más por la fuente Inter que exige el Design System; la hemos recortado al juego latino (de 72 KB a 48 KB) y queda en caché. Las visitas siguientes solo descargan el HTML.

15. Git y documentación

El historial del repositorio refleja la evolución del frontend:

Commit	Autor	Mensaje
b04bba5	dzeledonj-create	separacion del html y el php
c085557	dzeledonj-create	datos estaticos en el html conectado con el css
fee36fe	dsanchezg-campus	reestructura de carpeta config y public
c6fd987	dsanchezg-campus	reestructura de carpeta para los datos
510bac8	dsanchezg-campus	index como entrada a la web
2c6fe9b	mlizanoj	Eliminacion de archivos de logica antigua de php y js
9b3b390	dzeledonj-create	cambio de html y scss y la instalacion de nodejs para compilar el scss con packaje,json
90f2ad8	mlizanoj	Add supplier management pages: create_supplier.html and suppliers.html (reorganiza las vistas en carpetas y renombra partials → parciales)
141f04f · 79c4b94	dsanchezg-campus	Cambio a estructura MVC para views/ y resources/
4eda9a1	dzeledonj-create	ajustes del scss y agregacion de stylelintrc.json y lock.json para la prueba de prioridad de estilo inline

Después del cambio a la estructura de Laravel, los recursos que carga el navegador volvieron a `public/`, el SCSS pasó a `resources/scss/` y se eliminó `resources/js/menu.js` (JavaScript del proyecto PHP antiguo que ninguna vista usaba). Estos cambios y los de la revisión se suben en commits separados. El trabajo se integra en el repositorio del equipo mediante una pull request:

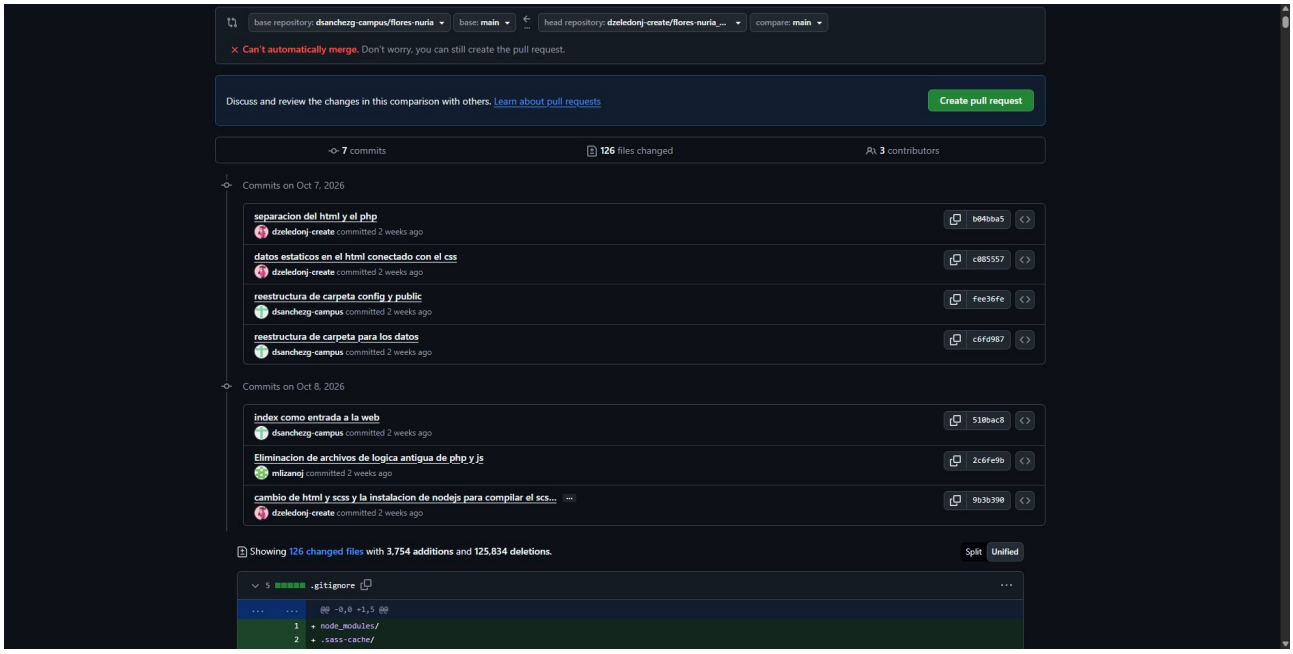


Figura 35. Pull request desde nuestro fork hacia el repositorio del equipo (dsanchezg-campus/flores-nuria): commits de las 3 personas del equipo.

`README.md` explica cómo ejecutar, compilar y validar; `CHANGELOG.md` recoge lo añadido, cambiado y eliminado en cada versión.

16. Relación con la rúbrica

CE	Indicadores	Evidencia	Apartado
CE2A	HTML estructurado · presentación CSS · separación · coherencia	19 vistas semánticas, 7 elementos nativos restilizados, 0 style/<style>/<script>, misma plantilla en todas	3
CE2B	Inline de prueba · cascada · versión externa · conclusión	26 declaraciones inline, 4 casos de prioridad con DevTools, versión con 4 clases, conclusión	4
CE2C	Globales · vinculación · organización · mantenimiento	55 tokens, 19/19 vistas enlazadas, 23 parciales, un token cambia todas las vistas	5
CE2D	Alternativa real · mismo HTML · cobertura · activación	theme-dark.css como alternate stylesheet, 4 formas de activar, axe 0 y 0 desbordes	6
CE2E	Cascada/herencia · redefinición · especificidad · depuración	Fallo real localizado en DevTools y corregido; tone(); 2 ! important justificados	7
CE2F	Propiedades · distribución · comprensión · diseño	Tabla por categorías, Grid/Flexbox justificado, 5 combinaciones explicadas	8
CE2G	Definición · reutilización · variantes · mantenibilidad	BEM, 12 componentes reutilizados, mixins de variantes	9
CE2H	Herramienta · detección · corrección · revalidación	W3C + Stylelint: 9 errores corregidos, advertencias justificadas	10
CE2I	Aplicación · actualización · componentes · utilidad	Guía v1.1 en HTML con el CSS real, 8 cambios justificados, pasos para ampliar	11

17. Conclusión

El frontend de Flores Nuria es ahora una pieza independiente: HTML semántico, hojas externas ordenadas por responsabilidad, una hoja alternativa que cambia todo el aspecto sin tocar el marcado y una guía de estilo que se mantiene sola porque usa el mismo CSS. Validar con dos herramientas nos hizo encontrar y corregir errores que el validador oficial no detecta.

Para los próximos hitos quedan las pruebas en Firefox, Edge y Safari y en un móvil real (H7), la auditoría de accesibilidad completa (H8) y las pruebas de usabilidad con usuarios (H9).